RTL Design Sherpa

GAXI BFM CocoTB Verification Framework — Rev 0.5.0

July 2026

Table of Contents

List of Tables

No tables in this document.

1 gaxi_command_handler.py

Enhanced GAXI command handler for transaction processing with sequential data generation and improved field extraction. This module provides a comprehensive command handler that bridges master and slave components, supporting both forwarding mode and response generation mode.

1.1 Overview

The `gaxi_command_handler.py` module provides the `GAXICommandHandler` class, which manages transaction processing between GAXI masters and slaves. It offers two operational modes: forwarding mode (master→slave) and response generation mode (slave→master), with enhanced sequential data generation for predictable testing.

1.1.1 Key Features

- **Dual operational modes**: Forwarding and response generation
- **Sequential data generation** for predictable read responses
- **Enhanced field extraction** supporting both APB and GAXI style packets
- **Memory model integration** with automatic fallback to sequential data
- **Comprehensive error handling** and recovery
- **Transaction dependency tracking** and ordering
- **Performance statistics** and latency measurement

1.2 Core Class

1.2.1 GAXICommandHandler

Enhanced command handler for GAXI transactions with comprehensive transaction processing capabilities.

1.2.1.1 Constructor

```
GAXICommandHandler(master, slave, memory_model=None, log=None,
                   response_generation_mode=False, **kwargs)
```

Parameters: - `master`: Master component instance - `slave`: Slave component instance - `memory_model`: Optional memory model for transactions (uses base MemoryModel) - `log`: Logger instance (defaults to master's logger) - `response_generation_mode`: If True, generate responses; if False, forward transactions - `**kwargs`: Additional configuration options

```
# Forwarding mode (master → slave)
handler = GAXICommandHandler(
    master=gaxi_master,
    slave=gaxi_slave,
    memory_model=memory_model,
```

```
    log=log,
    response_generation_mode=False
)

# Response generation mode (slave → master)
handler = GAXICommandHandler(
    master=gaxi_master,
    slave=gaxi_slave,
    memory_model=memory_model,
    log=log,
    response_generation_mode=True
)
```

1.2.1.2 Core Properties

- `master`: Master component
- `slave`: Slave component
- `memory_model`: Memory model for transactions
- `response_generation_mode`: Current operational mode
- `sequential_data_counter`: Counter for sequential data generation
- `pending_transactions`: Dictionary of pending transactions
- `completed_transactions`: Dictionary of completed transactions
- `pending_responses`: Dictionary of pending responses (response mode)
- `generated_responses`: Dictionary of generated responses (response mode)

1.3 Lifecycle Management

1.3.1 async start()

Start the command handler processing task.

Returns: Self for chaining

```
# Start the handler
await handler.start()
```

1.3.2 async stop()

Stop the command handler processing task.

Returns: Self for chaining

```
# Stop the handler
await handler.stop()
```

1.4 Transaction Processing (Forwarding Mode)

1.4.1 `async _forward_transactions()`

Forward transactions from master to slave with dependency tracking.

Internal method - called automatically in forwarding mode

1.4.2 `_is_dependency_satisfied(txn_id)`

Check if a transaction's dependencies are satisfied.

Parameters: - `txn_id`: Transaction ID to check

Returns: True if dependencies are satisfied

1.4.3 `async _send_to_slave(txn_id)`

Send a transaction to the slave component.

Parameters: - `txn_id`: Transaction ID to send

1.5 Response Generation (Response Mode)

1.5.1 `async _generate_response_for_transaction(cmd_transaction)`

Generate a response for a received command transaction with sequential data generation.

Parameters: - `cmd_transaction`: Command transaction to respond to

Features: - **Sequential data generation** for reads when memory is empty - **Memory integration** with automatic fallback - **Field extraction** supporting multiple packet formats - **Error handling** with comprehensive logging

```
# This method is called automatically in response generation mode
# when slave receives transactions
```

1.5.2 `_generate_sequential_data(address)`

Generate sequential data based on address for predictable testing.

Parameters: - `address`: Address being read

Returns: Sequential data value

Algorithm: - Combines sequential counter with address for predictability - Uses word-aligned addressing (address >> 2) - Maintains 32-bit data range - Increments counter for each generation

```
# Generates predictable data patterns:
# Address 0x1000 → 0x10000400 (first read)
```

```
# Address 0x1004 → 0x10000401 (next read)
# Address 0x1000 → 0x10000800 (later read)
```

1.6 Field Extraction and Response Handling

1.6.1 _extract_field_value(transaction, field_name, alt_field_name=None, default=0)

Extract a field value from a transaction supporting both storage methods.

Parameters: - `transaction`: Transaction object - `field_name`: Primary field name - `alt_field_name`: Alternative field name - `default`: Default value if field not found

Returns: Field value or default

Supports: - GAXI-style fields dictionary - APB-style attributes - Lowercase field names - Alternative field names

```
# Extracts from multiple field formats:
pwrite = handler._extract_field_value(transaction, 'pwrite',
'cmd_pwrite')
address = handler._extract_field_value(transaction, 'paddr',
'cmd_paddr')
data = handler._extract_field_value(transaction, 'pwdata',
'cmd_pwdata')
```

1.6.2 _set_response_field(response_packet, field_name, alt_field_name=None, value=0)

Set a field value in a response packet supporting both storage methods.

Parameters: - `response_packet`: Response packet to modify - `field_name`: Primary field name - `alt_field_name`: Alternative field name - `value`: Value to set

```
# Sets response fields in multiple formats:
handler._set_response_field(response_packet, 'prdata', 'rsp_prdata',
read_data)
handler._set_response_field(response_packet, 'pslverr', 'rsp_pslverr',
0)
```

1.7 Memory Operations

1.7.1 async _handle_memory_write(address, data, strobe)

Handle memory write operation with error handling.

Parameters: - `address`: Target address - `data`: Data to write - `strobe`: Write strobe mask

Returns: Success status

```
# Memory write with proper address masking and error handling
success = await handler._handle_memory_write(0x1000, 0xDEADBEEF, 0xF)
```

1.7.2 async _handle_memory_read(address)

Handle memory read operation with error handling.

Parameters: - `address`: Address to read from

Returns: Tuple of (success, data)

```
# Memory read with automatic fallback to sequential data
success, data = await handler._handle_memory_read(0x1000)
if not success:
    # Will use sequential data generation
    pass
```

1.8 Sequence Processing

1.8.1 async process_sequence(sequence)

Process a GAXI sequence through the master-slave connection.

Parameters: - `sequence`: GAXISequence to process

Returns: Dictionary of responses by transaction index

Features: - **Dependency tracking**: Handles transaction dependencies - **Completion monitoring**: Waits for transaction completion - **Response mapping**: Maps responses to sequence indexes

```
# Process sequence with dependencies
sequence = GAXISequence("test_sequence")
sequence.add_data_transaction(0x1000)
sequence.add_data_transaction(0x2000, depends_on=0)

response_map = await handler.process_sequence(sequence)
print(f"Response 0: {response_map[0]}")
print(f"Response 1: {response_map[1]}")
```

1.9 Statistics and Monitoring

1.9.1 get_stats()

Get comprehensive handler statistics.

Returns: Dictionary with statistics including: - Transaction counts and timing - Memory operation statistics - Sequential data generation stats - Component statistics - Error tracking

```
stats = handler.get_stats()
print(f"Completed transactions: {stats['completed_transactions']}")
print(f"Sequential reads: {stats['sequential_reads']}")
print(f"Memory operations: {stats['memory_operations']}")
print(f"Average latency: {stats['avg_latency']} ns")
```

1.9.2 get_transaction_status(txn_id=None)

Get status of specific transaction or all transactions.

Parameters: - `txn_id`: Transaction ID to check (None for all)

Returns: Transaction status information

```
# Get status of all transactions
status = handler.get_transaction_status()
print(f"Pending: {status['pending']}")
print(f"Completed: {status['completed']}")

# Get status of specific transaction
specific_status = handler.get_transaction_status(txn_id)
```

1.9.3 reset()

Reset the command handler to initial state.

```
# Reset all tracking and statistics
handler.reset()
```

1.10 Usage Patterns

1.10.1 Basic Forwarding Setup

```
async def setup_forwarding_handler():
    """Set up command handler for master→slave forwarding"""

    # Create memory model
    memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)

    # Create handler in forwarding mode
    handler = GAXICommandHandler(
        master=gaxi_master,
        slave=gaxi_slave,
        memory_model=memory,
        log=log,
        response_generation_mode=False
    )

    # Start processing
```

```
    await handler.start()

    return handler

async def test_forwarding():
    handler = await setup_forwarding_handler()

    # Send transactions through master
    for i in range(10):
        packet = gaxi_master.create_packet(addr=0x1000 + i*4,
data=i*0x100)
        await gaxi_master.send(packet)

    # Wait for completion
    while handler.get_transaction_status()['pending'] > 0:
        await RisingEdge(dut.clk)

    # Get statistics
    stats = handler.get_stats()
    log.info(f"Forwarding test completed: {stats}")

    await handler.stop()
```

1.10.2 Response Generation Setup

```
async def setup_response_handler():
    """Set up command handler for slave→master response generation"""

    # Create memory model with initial data
    memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)

    # Populate memory with test data
    for addr in range(0, 0x1000, 4):
        data = bytearray([(addr + i) & 0xFF for i in range(4)])
        memory.write(addr, data)

    # Create handler in response generation mode
    handler = GAXICommandHandler(
        master=gaxi_master,
        slave=gaxi_slave,
        memory_model=memory,
        log=log,
        response_generation_mode=True
    )

    await handler.start()
    return handler
```

```
async def test_response_generation():
    handler = await setup_response_handler()

    # Send commands to slave
    for i in range(10):
        # Write command
        write_cmd = create_write_command(addr=0x1000 + i*4,
data=i*0x100)
        gaxi_slave._recvQ.append(write_cmd)

        # Read command
        read_cmd = create_read_command(addr=0x1000 + i*4)
        gaxi_slave._recvQ.append(read_cmd)

    # Wait for responses
    await Timer(1000, units='ns')

    # Check response statistics
    stats = handler.get_stats()
    log.info(f"Generated {stats['generated_responses']} responses")
    log.info(f"Sequential reads: {stats['sequential_reads']}")

    await handler.stop()
```

1.10.3 Advanced Dependency Testing

```
async def test_dependency_handling():
    """Test transaction dependency handling"""

    handler = await setup_forwarding_handler()

    # Create sequence with dependencies
    sequence = GAXISequence("dependency_test")

    # Transaction 0: Base transaction
    sequence.add_data_transaction(0x1000, delay=0)

    # Transaction 1: Depends on transaction 0
    sequence.add_data_transaction(0x2000, delay=0, depends_on=0)

    # Transaction 2: Also depends on transaction 0
    sequence.add_data_transaction(0x3000, delay=0, depends_on=0)

    # Transaction 3: Depends on transaction 1
    sequence.add_data_transaction(0x4000, delay=0, depends_on=1)

    # Process sequence with dependency resolution
    response_map = await handler.process_sequence(sequence)
```

```
    # Verify responses
    assert len(response_map) == 4
    log.info("Dependency test completed successfully")

    # Check dependency statistics
    stats = handler.get_stats()
    assert stats['dependency_violations'] == 0

    await handler.stop()
```

1.10.4 Memory Integration Testing

```
async def test_memory_integration():
    """Test memory model integration with fallback"""

    # Create memory with limited data
    memory = MemoryModel(num_lines=64, bytes_per_line=4, log=log)

    # Only populate some addresses
    for addr in range(0x0000, 0x0100, 4):
        data = bytearray([addr & 0xFF, (addr >> 8) & 0xFF, 0x00,
0x00])
        memory.write(addr, data)

    handler = GAXICommandHandler(
        master=gaxi_master,
        slave=gaxi_slave,
        memory_model=memory,
        log=log,
        response_generation_mode=True
    )

    await handler.start()

    # Test memory reads (should use memory data)
    memory_read_cmd = create_read_command(addr=0x0010)
    gaxi_slave._recvQ.append(memory_read_cmd)

    # Test unmapped reads (should use sequential data)
    sequential_read_cmd = create_read_command(addr=0x8000)
    gaxi_slave._recvQ.append(sequential_read_cmd)

    await Timer(1000, units='ns')

    # Check statistics
    stats = handler.get_stats()
    log.info(f"Memory reads: {stats['memory_reads']}")
    log.info(f"Sequential reads: {stats['sequential_reads']}")
```

```
    await handler.stop()
```

1.10.5 Performance Monitoring

```
class PerformanceMonitor:
    def __init__(self, handler):
        self.handler = handler
        self.monitoring = True

    async def monitor_performance(self):
        """Continuously monitor handler performance"""
        while self.monitoring:
            await Timer(1000000, units='ns')  # Every 1ms

            stats = self.handler.get_stats()

            # Check performance metrics
            if stats['completed_transactions'] > 0:
                avg_latency = stats['avg_latency']
                if avg_latency > 10000:  # > 10µs
                    log.warning(f"High latency detected:
{avg_latency:.1f}ns")

            # Check error rates
            if stats.get('dependency_violations', 0) > 0:
                log.warning(f"Dependency violations:
{stats['dependency_violations']}")

            # Report throughput
            if stats['completed_transactions'] % 100 == 0:
                log.info(f"Processed {stats['completed_transactions']}
transactions")

async def test_with_monitoring():
    handler = await setup_forwarding_handler()
    monitor = PerformanceMonitor(handler)

    # Start monitoring
    monitor_task = cocotb.start_soon(monitor.monitor_performance())

    # Run test
    await run_high_throughput_test(handler)

    # Stop monitoring
    monitor.monitoring = False
    monitor_task.kill()

    await handler.stop()
```

1.11 Error Handling and Recovery

1.11.1 Automatic Error Recovery

```
try:
    # Process transaction with automatic error handling
    await handler._generate_response_for_transaction(transaction)
except Exception as e:
    # Handler automatically:
    # - Logs detailed error information
    # - Updates error statistics
    # - Continues processing other transactions
    # - Maintains system stability
    pass
```

1.11.2 Field Extraction Fallbacks

```
# Handler automatically tries multiple field extraction methods:
# 1. Fields dictionary (GAXI style)
# 2. Direct attributes (APB style)
# 3. Alternative field names
# 4. Lowercase versions
# 5. Returns default value if none found

value = handler._extract_field_value(
    transaction,
    'paddr',           # Primary name
    'cmd_paddr',       # Alternative name
    default=0x0        # Safe default
)
```

1.11.3 Memory Operation Fallbacks

```
# For read operations:
# 1. Try memory model read
# 2. On failure, generate sequential data
# 3. Log the fallback for debugging
# 4. Continue operation seamlessly

success, data = await handler._handle_memory_read(address)
if not success:
    # Automatic fallback to sequential data generation
    data = handler._generate_sequential_data(address)
```

1.12 Integration with Other Components

1.12.1 Master Integration

```
# Handler integrates seamlessly with GAXIMaster
master = GAXIMaster(dut, "TestMaster", "", clock, field_config)
handler = GAXICommandHandler(master, slave,
response_generation_mode=False)

# Transactions from master are automatically forwarded
await master.send(packet)
```

1.12.2 Slave Integration

```
# Handler monitors slave receive queue for transactions
slave = GAXISlave(dut, "TestSlave", "", clock, field_config)
handler = GAXICommandHandler(master, slave,
response_generation_mode=True)

# Received transactions automatically generate responses
```

1.12.3 Memory Model Integration

```
# Uses base MemoryModel directly for maximum compatibility
memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)
handler = GAXICommandHandler(master, slave, memory_model=memory)

# Automatic memory operations with proper error handling
```

1.13 Best Practices

1.13.1 1. Choose Appropriate Mode

```
# Use forwarding mode for master→slave testing
handler = GAXICommandHandler(master, slave,
response_generation_mode=False)

# Use response generation mode for slave→master testing
handler = GAXICommandHandler(master, slave,
response_generation_mode=True)
```

1.13.2 2. Initialize Memory Model

```
# Always provide memory model for realistic testing
memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)
handler = GAXICommandHandler(master, slave, memory_model=memory)
```

1.13.33. Monitor Statistics

```
# Regular statistics monitoring
async def monitor_handler():
    while True:
        await Timer(1000000, units='ns')
        stats = handler.get_stats()
        if stats['error_count'] > 0:
            log.warning(f"Errors detected: {stats}")
```

1.13.44. Handle Dependencies Properly

```
# Always validate dependencies in sequences
sequence.validate_dependencies()
response_map = await handler.process_sequence(sequence)
```

1.13.55. Use Sequential Data for Testing

```
# Sequential data generation provides predictable test patterns
# Address-based: Different addresses generate different patterns
# Counter-based: Each read increments the pattern
# Useful for debugging and verification
```

The GAXICommandHandler provides a comprehensive solution for GAXI transaction processing with enhanced data generation, robust error handling, and flexible operational modes for comprehensive verification scenarios.

2 gaxi_component_base.py

Unified base class for all GAXI components that consolidates common functionality across GAXIMaster, GAXIMonitor, and GAXISlave, eliminating code duplication while preserving exact APIs and timing.

2.1 Overview

The `GAXIComponentBase` class provides: - **Unified signal resolution** and data handling setup - **Shared field configuration** handling and packet management
- **Memory model integration** using base MemoryModel directly - **Common statistics** and logging patterns - **Resolved signal passing** to DataStrategies eliminating guesswork

All existing parameters are preserved and used exactly as before, with the addition of optional manual signal mapping.

2.2 Class

2.2.1 GAXIComponentBase

```
class GAXIComponentBase:
    def __init__(self, dut, title, prefix, clock, field_config,
                 protocol_type,  # Must be specified by subclass
                 mode='skid', bus_name='', pkt_prefix='',
multi_sig=False,
                 randomizer=None, memory_model=None, log=None,
                 super_debug=False, signal_map=None,
packet_class=None, **kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title/name - `prefix`: Bus prefix - `clock`: Clock signal - `field_config`: Field configuration (FieldConfig or dict) - `protocol_type`: Protocol type ('gaxi_master', 'gaxi_slave', 'axis_master', 'axis_slave', 'fifo_master', or 'fifo_slave') - `mode`: GAXI mode ('skid', 'fifo_mux', 'fifo_flop') - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `multi_sig`: Whether using multi-signal mode - `randomizer`: Optional randomizer for timing - `memory_model`: Optional memory model for transactions - `log`: Logger instance - `super_debug`: Enable detailed debugging - `signal_map`: Optional dict mapping signal names to DUT signal names - `packet_class`: Optional `Packet` subclass produced by the component's pipeline. `None` (default) keeps the class-level default (`GAXIPacket` for GAXI, `FIFOPacket` for FIFO). Must be a `Packet` subclass — validated at construction. See `_build_packet()`. - `**kwargs`: Additional arguments for specific component types

Signal Map Format:

```
# Manual signal mapping (bypasses automatic discovery)
signal_map = {
    'valid': 'dut_valid_signal_name',
    'ready': 'dut_ready_signal_name',
    'data': 'dut_data_signal_name'     # For single-signal mode
    # Or individual field names for multi-signal mode
}
```

2.3 Key Methods

2.3.1 Core Initialization

2.3.1.1 `complete_base_initialization(bus=None)`

Complete initialization after cocotb parent class setup.

Must be called by subclasses after their cocotb parent class (BusDriver/BusMonitor) initialization is complete.

```
# In subclass __init__ after BusDriver.__init__:
self.complete_base_initialization(self.bus)
```

2.3.2 Unified Data Operations

2.3.2.1 get_data_dict_unified()

Get current data from signals with automatic field unpacking.

Returns: Dictionary of field values, properly unpacked

```
# Clean data collection with automatic field unpacking
data = component.get_data_dict_unified()
print(data)  # {'addr': 0x1000, 'data': 0xDEADBEEF, 'cmd': 0x2}
```

2.3.2.2 drive_transaction_unified(transaction)

Drive transaction data using unified DataDrivingStrategy.

Parameters: - `transaction`: Transaction to drive

Returns: True if successful, False otherwise

```
packet = component.create_packet(addr=0x1000, data=0xDEADBEEF)
success = component.drive_transaction_unified(packet)
```

2.3.2.3 clear_signals_unified()

Clear all data signals using unified strategy.

```
component.clear_signals_unified()  # Set all signals to 0
```

2.3.3 Memory Operations

2.3.3.1 write_to_memory_unified(transaction)

Write transaction to memory using base MemoryModel.

Parameters: - `transaction`: Transaction to write

Returns: Tuple of (success, error_message)

```
success, error = component.write_to_memory_unified(packet)
if not success:
    log.error(f"Memory write failed: {error}")
```

2.3.3.2 read_from_memory_unified(transaction, update_transaction=True)

Read data from memory using base MemoryModel.

Parameters: - `transaction`: Transaction with address to read - `update_transaction`: Whether to update transaction with read data

Returns: Tuple of (success, data, error_message)

```
success, data, error = component.read_from_memory_unified(packet)
if success:
    log.info(f"Read data: 0x{data:X}")
```

2.3.4 Packet Construction

*2.3.4.1 _build_packet(**field_values)*

The single extension point for packet construction across the GAXI and FIFO component families. Every packet produced by the receive pipeline, `create_packet()`, and the master transmit path comes from this hook.

Parameters: - `**field_values`: Optional initial field values. Names matching a field the packet exposes are assigned after construction; unknown names are ignored (preserving the historical `create_packet()` contract).

Returns: A newly constructed packet instance.

The class constructed is resolved in this order:

1. `self.packet_class`, if a `packet_class=` argument was passed to the component or its factory.
2. `self._default_packet_class` — `GAXIPacket` for GAXI components, `FIFOPacket` for the FIFO chassis.

With neither supplied, behavior is identical to before the hook existed: a plain `GAXIPacket(self.field_config)`.

```
# Default: plain GAXIPacket
packet = component._build_packet()

# Via the packet_class argument (also accepted by every factory)
slave = create_gaxi_slave(dut, "Slave", "", clock,
packet_class=MyPacket)
# slave's receive pipeline now produces MyPacket instances
```

2.3.4.1.1 Overriding the hook

`packet_class=` covers packet classes constructible as `PacketClass(field_config)`. When a protocol packet needs **extra constructor arguments**, override `_build_packet()` instead — this is the supported extension point for protocol BFMs that delegate to the GAXI pipelines (AXI4/AXI5/AXIS/FIFO):

```
class AXIS5Slave(GAXISlave):
    def __init__(self, *args, parity_enabled=False, **kwargs):
        super().__init__(*args, **kwargs)
        self.parity_enabled = parity_enabled
```

```
    def _build_packet(self, **field_values):
        return AXIS5Packet(
            self.field_config,
            parity_enabled=self.parity_enabled,
            **field_values,
        )
```

Because the whole pipeline routes through the hook, downstream `isinstance(packet, AXIS5Packet)` checks keep working — previously the receive path hard-coded `GAXIPacket(self.field_config)`, so a delegating slave or monitor silently lost its protocol packet subclass.

An override takes precedence over `packet_class`. Subclasses that only need a different default (no extra constructor arguments) can instead set the class attribute:

```
class FIFOComponentBase(GAXIComponentBase):
    _default_packet_class = FIFOPacket
```

2.3.5 Statistics and Configuration

2.3.5.1 get_base_stats_unified()

Get comprehensive base statistics common to all components.

Returns: Dictionary containing base statistics

```
stats = component.get_base_stats_unified()
print(f"Component type: {stats['component_type']}")
print(f"Signal mapping source: {stats['signal_mapping_source']}")
print(f"Field count: {stats['field_count']}")
```

2.3.5.2 set_randomizer(randomizer)

Set new randomizer for timing control.

Parameters: - `randomizer`: FlexRandomizer instance

```
from CocoTBFramework.components.shared.flex_randomizer import
FlexRandomizer


new_randomizer = FlexRandomizer({
    'valid_delay': ([(0, 0), (1, 5)], [0.8, 0.2])
})
component.set_randomizer(new_randomizer)
```

2.4 Usage Patterns

2.4.1 Basic Component Creation

```
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster
from CocoTBFramework.components.shared.field_config import FieldConfig

# Create field configuration
field_config = FieldConfig()
field_config.add_field(FieldDefinition("addr", 32, format="hex"))
field_config.add_field(FieldDefinition("data", 32, format="hex"))

# Create master (inherits from GAXIComponentBase)
master = GAXIMaster(
    dut=dut,
    title="TestMaster",
    prefix="",
    clock=clock,
    field_config=field_config,
    mode='skid',
    multi_sig=True,  # Individual signals for each field
    log=log          # Required by GAXIMaster
)
```

2.4.2 Manual Signal Mapping

```
# For non-standard signal names
signal_map = {
    'valid': 'master_valid_custom',
    'ready': 'slave_ready_custom',
    'addr': 'address_signal',      # Multi-signal mode
    'data': 'data_signal'
}

master = GAXIMaster(
    dut=dut,
    title="CustomMaster",
    prefix="",
    clock=clock,
    field_config=field_config,
    signal_map=signal_map  # Override automatic discovery
)
```

2.4.3 Memory Integration

```
from CocoTBFramework.components.shared.memory_model import MemoryModel

# Create memory model
memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)
```

```python
# Create component with memory
master = GAXIMaster(
    dut=dut,
    title="MemoryMaster",
    prefix="",
    clock=clock,
    field_config=field_config,
    memory_model=memory
)

# Write to memory through component
packet = master.create_packet(addr=0x1000, data=0xDEADBEEF)
success, error = master.write_to_memory_unified(packet)
```

2.4.4 Unified Data Collection

```python
class CustomMonitor(GAXIComponentBase, BusMonitor):
    def __init__(self, dut, title, prefix, clock, field_config):
        GAXIComponentBase.__init__(
            self, dut, title, prefix, clock, field_config,
            protocol_type='gaxi_master'  # Monitor master side
        )
        BusMonitor.__init__(self, dut, prefix, clock)
        self.complete_base_initialization(self.bus)

    @cocotb.coroutine
    def _monitor_recv(self):
        while True:
            yield RisingEdge(self.clock)

            # Use unified data collection
            data = self.get_data_dict_unified()

            if data.get('valid', 0) == 1:
                # Process transaction
                packet = self.create_packet(**data)
                self._recv(packet)  # Add to cocotb queue
```

2.4.5 Advanced Statistics

```python
def analyze_component_performance(component):
    """Analyze component performance using base stats"""
    stats = component.get_base_stats_unified()

    print(f"=== Component Analysis: {stats['title']} ===")
    print(f"Type: {stats['component_type']}")
    print(f"Mode: {stats['mode']}")
    print(f"Multi-signal: {stats['multi_signal']}")
```

```
    print(f"Fields: {stats['field_count']}")

    # Signal resolver statistics
    if 'signal_resolver_stats' in stats:
        resolver_stats = stats['signal_resolver_stats']
        print(f"Signal resolution rate:
{resolver_stats['resolution_rate']:.1f}%")
        print(f"Signal conflicts: {resolver_stats['conflicts']}")

    # Data strategy statistics
    if 'data_collector_stats' in stats:
        collector_stats = stats['data_collector_stats']
        print(f"Data collection mode: {collector_stats['mode']}")
        print(f"Performance optimized:
{collector_stats['performance_optimized']}")

    # Memory statistics
    if 'memory_stats' in stats:
        memory_stats = stats['memory_stats']
        print(f"Memory operations: {memory_stats['reads'] +
memory_stats['writes']}")
        print(f"Memory coverage: {memory_stats['read_coverage']:.1%}")
```

2.5 Error Handling

2.5.1 Signal Resolution Errors

```
try:
    master = GAXIMaster(dut, "Master", "", clock, field_config)
except RuntimeError as e:
    # Detailed error with signal mapping diagnostics
    log.error(f"Signal mapping failed: {e}")

    # Try manual mapping as fallback
    signal_map = create_fallback_signal_map(dut)
    master = GAXIMaster(dut, "Master", "", clock, field_config,
                        signal_map=signal_map)
```

2.5.2 Memory Operation Errors

```
# Memory operations return success/error tuples
success, error = component.write_to_memory_unified(packet)
if not success:
    if "boundary" in error.lower():
        log.error(f"Address out of bounds: {error}")
    elif "type" in error.lower():
        log.error(f"Data type error: {error}")
    else:
        log.error(f"Memory error: {error}")
```

2.6 Internal Architecture

2.6.1 Signal Resolution Flow

1. **SignalResolver creation** with protocol type and parameters
2. **Pattern matching** against DUT ports (or manual mapping)
3. **Signal validation** and conflict detection
4. **DataStrategy creation** with resolved signals
5. **Component signal application** via `apply_to_component()`

2.6.2 Data Strategy Integration

```
# Internal flow (handled automatically)
resolver = SignalResolver(protocol_type, dut, bus, log, title, ...)
resolved_signals = resolver.resolved_signals

# Pass resolved signals to strategies (eliminates guesswork)
data_collector = DataCollectionStrategy(...,
resolved_signals=resolved_signals)
data_driver = DataDrivingStrategy(...,
resolved_signals=resolved_signals)
```

2.6.3 Memory Model Integration

- Uses base MemoryModel methods directly
- Transaction-based read/write operations
- Automatic field extraction and validation
- Error handling with detailed messages

2.7 Best Practices

2.7.1 1. Use Automatic Signal Discovery First

```
# Try automatic discovery
component = GAXIMaster(dut, title, prefix, clock, field_config)

# Fall back to manual mapping if needed
if not all_signals_found:
    signal_map = {...}
    component = GAXIMaster(..., signal_map=signal_map)
```

2.7.2 2. Always Complete Base Initialization

```
# In subclass after cocotb parent initialization
class MyComponent(GAXIComponentBase, BusDriver):
    def __init__(self, ...):
```

```
        GAXIComponentBase.__init__(self, ...)
        BusDriver.__init__(self, ...)
        self.complete_base_initialization(self.bus)  # Essential!
```

2.7.3 3. Use Unified Methods for Consistency

```
# Prefer unified methods over custom implementations
data = component.get_data_dict_unified()  # Not custom
_get_data_dict()
success = component.drive_transaction_unified(packet)  # Not custom
driving
```

2.7.4 4. Monitor Statistics for Performance

```
# Regular performance monitoring
stats = component.get_base_stats_unified()
if 'data_collector_stats' in stats:
    collector_stats = stats['data_collector_stats']
    if not collector_stats['performance_optimized']:
        log.warning("Data collection not optimized")
```

2.7.5 5. Handle Memory Operations Gracefully

```
# Always check memory operation results
success, error = component.write_to_memory_unified(packet)
if not success:
    # Handle error appropriately for test context
    handle_memory_error(error, packet)
```

GAXIComponentBase provides the foundation for all GAXI components, ensuring consistent behavior, optimal performance, and comprehensive error handling across the entire GAXI ecosystem.

3 gaxi_factories.py

Updated GAXI factories with fixed parameter handling to match TB instantiation patterns. This module provides simplified factory functions for creating GAXI components with proper defaults and consistent parameter handling across all factory methods.

3.1 Overview

The `gaxi_factories.py` module provides factory functions for creating GAXI components (Master, Slave, Monitor, Scoreboard) with simplified configuration and proper parameter defaults. All existing APIs are preserved and parameters are passed through exactly as before, with enhanced defaults for signal prefixes and naming.

3.1.1 Key Features

- **Simplified component creation** with sensible defaults

- **Consistent parameter handling** across all factories
- **Backward compatibility** - all existing parameters preserved
- **Enhanced defaults** for signal prefixes (empty strings work for most cases)
- **Memory model integration** using base MemoryModel directly
- **Complete system creation** with factory methods for full environments

3.2 Field Configuration

3.2.1 `get_default_field_config(data_width=32)`

Get standard field configuration for GAXI protocol.

Parameters: - `data_width`: Data width in bits (default: 32)

Returns: FieldConfig object for standard data field

```
# Get default 32-bit data configuration
config = get_default_field_config()

# Get 64-bit data configuration
config_64 = get_default_field_config(data_width=64)
```

3.3 Component Factories

3.3.1 `create_gaxi_master()`

Create a GAXI Master component with simplified configuration.

```
create_gaxi_master(dut, title, prefix, clock, field_config=None,
packet_class=None,
                   randomizer=None, memory_model=None,
memory_fields=None, log=None,
                   signal_map=None, optional_signal_map=None,
field_mode=False,
                   multi_sig=False, mode='skid', bus_name='',
pkt_prefix='',
                   **kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title - `prefix`: Signal prefix - `clock`: Clock signal - `field_config`: Field configuration (default: standard data field) - `packet_class`: Packet class produced by the component's pipeline (`None` = `GAXIPacket`). Wired through the `_build_packet()` hook - `randomizer`: Timing randomizer (default: standard master constraints) - `memory_model`: Memory model for transactions (optional) - `memory_fields`: Field mapping for memory operations (unused - kept for compatibility) - `log`: Logger instance (default: dut's logger) - `signal_map`: Manual signal mapping dict (forwarded to the component; None = automatic signal discovery) - `optional_signal_map`: Optional signal mapping (unused - kept for

compatibility) - `field_mode`: Field mode (unused - kept for compatibility) - `multi_sig`: Whether using multi-signal mode - `mode`: Operating mode ('skid', 'fifo_mux', 'fifo_flop') - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `**kwargs`: Additional arguments

Returns: GAXIMaster instance

```
# Basic master creation
master = create_gaxi_master(
    dut=dut,
    title="TestMaster",
    prefix="master_",
    clock=dut.clk
)

# Advanced master with custom configuration
master = create_gaxi_master(
    dut=dut,
    title="AdvancedMaster",
    prefix="",
    clock=dut.clk,
    field_config=custom_field_config,
    memory_model=shared_memory,
    multi_sig=True,
    mode='fifo_flop',
    log=custom_logger
)
```

3.3.2 `create_gaxi_slave()`

Create a GAXI Slave component with simplified configuration.

```
create_gaxi_slave(dut, title, prefix, clock, field_config=None,
field_mode=False,
                  packet_class=None, randomizer=None,
memory_model=None,
                  memory_fields=None, log=None, mode='skid',
signal_map=None,
                  optional_signal_map=None, multi_sig=False,
bus_name='',
                  pkt_prefix='', **kwargs)
```

Parameters: Same as `create_gaxi_master()` with slave-specific defaults

Returns: GAXISlave instance

```
# Basic slave creation
slave = create_gaxi_slave(
    dut=dut,
    title="TestSlave",
    prefix="slave_",
```

```
    clock=dut.clk
)

# Slave with shared memory model
slave = create_gaxi_slave(
    dut=dut,
    title="MemorySlave",
    prefix="",
    clock=dut.clk,
    memory_model=shared_memory,
    mode='skid'
)
```

3.3.3 create_gaxi_monitor()

Create a GAXI Monitor component with simplified configuration.

```
create_gaxi_monitor(dut, title, prefix, clock, field_config=None,
is_slave=False,
                    log=None, mode='skid', bus_name='', pkt_prefix='',
                    multi_sig=False, signal_map=None,
packet_class=None, **kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title - `prefix`: Signal prefix - `clock`: Clock signal - `field_config`: Field configuration (default: standard data field) - `is_slave`: If True, monitor slave side; if False, monitor master side - `log`: Logger instance (default: dut's logger) - `mode`: Operating mode ('skid', 'fifo_mux', 'fifo_flop') - `multi_sig`: Whether using multi-signal mode - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `signal_map`: Manual signal mapping dict (forwarded to the component; None = automatic signal discovery) - `packet_class`: Packet class produced by the receive pipeline (`None` = `GAXIPacket`). Wired through the `_build_packet()` hook - `**kwargs`: Additional arguments

Returns: GAXIMonitor instance

```
# Master-side monitor
master_monitor = create_gaxi_monitor(
    dut=dut,
    title="MasterMonitor",
    prefix="",
    clock=dut.clk,
    is_slave=False
)

# Slave-side monitor
slave_monitor = create_gaxi_monitor(
    dut=dut,
    title="SlaveMonitor",
    prefix="",
    clock=dut.clk,
```

```
    is_slave=True,
    mode='fifo_flop'
)
```

3.3.4 Custom packet classes

Every component factory forwards `packet_class` into the component's `_build_packet()` hook, so the whole pipeline — receive path, `create_packet()`, and the master transmit path — produces your class instead of `GAXIPacket`:

```
class MyPacket(GAXIPacket):
    pass

slave = create_gaxi_slave(dut, "Slave", "", clock,
packet_class=MyPacket)
monitor = create_gaxi_monitor(dut, "Mon", "", clock,
packet_class=MyPacket)

# Or apply it to a whole component set at once
components = create_gaxi_components(dut, clock, packet_class=MyPacket)

pkt = monitor._recvQ.popleft()
assert isinstance(pkt, MyPacket)   # protocol subclass survives the
pipeline
```

`packet_class` covers classes constructible as `MyPacket(field_config)`. If your packet needs additional constructor arguments, subclass the component and override `_build_packet()` instead — see the component base docs.

Omitting `packet_class` is fully backward compatible: components keep producing plain `GAXIPacket` instances.

3.3.5 `create_gaxi_scoreboard()`

Create a GAXI Scoreboard with simplified configuration.

```
create_gaxi_scoreboard(name, field_config=None, log=None)
```

Parameters: - `name`: Scoreboard name - `field_config`: Field configuration (default: standard data field) - `log`: Logger instance

Returns: GAXIScoreboard instance

```
# Basic scoreboard
scoreboard = create_gaxi_scoreboard("TestScoreboard")

# Scoreboard with custom configuration
scoreboard = create_gaxi_scoreboard(
    name="CustomScoreboard",
```

```
    field_config=custom_field_config,
    log=custom_logger
)
```

3.4 System Creation Factories

3.4.1 `create_gaxi_components()`

Create a complete set of GAXI components (master, slave, monitors, scoreboard).

```
create_gaxi_components(dut, clock, title_prefix="", field_config=None,
                       field_mode=False, packet_class=None,
memory_model=None,
                       log=None, mode='skid', signal_map=None,
optional_signal_map=None,
                       multi_sig=False, bus_name='', pkt_prefix='',
                       **kwargs)
```

Parameters: - `dut`: Device under test - `clock`: Clock signal - `title_prefix`: Prefix for component titles - `field_config`: Field configuration (default: standard data field) - `field_mode`: Field mode (unused - kept for compatibility) - `packet_class`: Packet class produced by the component's pipeline (`None` = `GAXIPacket`). Wired through the `_build_packet()` hook - `memory_model`: Memory model for components (auto-created if None) - `log`: Logger instance - `mode`: Operating mode for slave/monitor - `signal_map`: Manual signal mapping dict (forwarded to the component; None = automatic signal discovery) - `optional_signal_map`: Optional signal mapping (unused - kept for compatibility) - `multi_sig`: Whether using multi-signal mode - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `**kwargs`: Additional configuration passed to all components

Returns: Dictionary containing all created components

```
# Create complete GAXI system
components = create_gaxi_components(
    dut=dut,
    clock=dut.clk,
    title_prefix="GAXI_"
)

# Access components
master = components['master']
slave = components['slave']
master_monitor = components['master_monitor']
slave_monitor = components['slave_monitor']
scoreboard = components['scoreboard']
memory_model = components['memory_model']
```

3.4.2 create_gaxi_system()

Create a complete GAXI system with all components - alias for `create_gaxi_components()`.

```
create_gaxi_system(dut, clock, title_prefix="", field_config=None,
                   memory_model=None, log=None, bus_name='',
pkt_prefix='',
                   **kwargs)
```

Parameters: Simplified version of `create_gaxi_components()` parameters

Returns: Dictionary containing all created components and shared resources

```
# Create GAXI system with clean API
system = create_gaxi_system(
    dut=dut,
    clock=dut.clk,
    title_prefix="System_",
    field_config=custom_config
)
```

3.4.3 create_gaxi_test_environment()

Create a complete GAXI test environment ready for immediate use.

```
create_gaxi_test_environment(dut, clock, bus_name='', pkt_prefix='',
**kwargs)
```

Parameters: - `dut`: Device under test - `clock`: Clock signal - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `**kwargs`: Configuration overrides including: - `title_prefix`: Component title prefix (default: 'GAXI_') - `field_config`: Field configuration - `data_width`: Data width (default: 32) - `memory_size`: Memory size in lines (default: 1024) - `log`: Logger instance

Returns: Dictionary with complete test environment including convenience functions

```
# Create ready-to-use test environment
env = create_gaxi_test_environment(
    dut=dut,
    clock=dut.clk,
    data_width=64,
    memory_size=2048
)

# Use convenience functions
await env['send_data'](0xDEADBEEF)
data = env['read_memory'](0x1000)
env['write_memory'](0x1000, 0x12345678)
all_stats = env['get_all_stats']()
```

3.5 Usage Patterns

3.5.1 Basic Component Creation

```
@cocotb.test()
async def test_basic_gaxi(dut):
    """Basic GAXI test with factory-created components"""

    # Create individual components
    master = create_gaxi_master(
        dut=dut,
        title="TestMaster",
        prefix="",
        clock=dut.clk
    )

    slave = create_gaxi_slave(
        dut=dut,
        title="TestSlave",
        prefix="",
        clock=dut.clk
    )

    # Test basic functionality
    await master.reset_bus()
    await slave.reset_bus()

    # Send test transaction
    packet = master.create_packet(data=0xDEADBEEF)
    await master.send(packet)

    # Verify reception
    await Timer(100, units='ns')
    received = slave.get_observed_packets()
    assert len(received) > 0
```

3.5.2 Complete System Creation

```
@cocotb.test()
async def test_complete_system(dut):
    """Test with complete GAXI system"""

    # Create complete system
    # (data_width/memory_size are create_gaxi_test_environment
options;
    #  create_gaxi_system takes a field_config and memory_model
instead)
    system = create_gaxi_system(
        dut=dut,
```

```
        clock=dut.clk,
        title_prefix="Test_",
        log=log
    )

    # Extract components
    master = system['master']
    slave = system['slave']
    master_monitor = system['master_monitor']
    slave_monitor = system['slave_monitor']
    scoreboard = system['scoreboard']
    memory = system['memory_model']

    # Connect monitors to scoreboard
    master_monitor.add_callback(scoreboard.add_expected)
    slave_monitor.add_callback(scoreboard.add_actual)

    # Run test sequence
    await run_test_sequence(master, slave)

    # Check results
    stats = {
        'master': master.get_stats(),
        'slave': slave.get_stats(),
        'master_monitor': master_monitor.get_stats(),
        'slave_monitor': slave_monitor.get_stats(),
        'memory': memory.get_stats()
    }

    log.info(f"Test completed: {stats}")
```

3.5.3 Test Environment Creation

```
@cocotb.test()
async def test_with_environment(dut):
    """Test using complete test environment"""

    # Create test environment with convenience functions
    env = create_gaxi_test_environment(
        dut=dut,
        clock=dut.clk,
        title_prefix="ENV_",
        data_width=64,
        memory_size=2048
    )

    # Use convenience functions for testing
    test_data = [0xDEADBEEF, 0xCAFEBABE, 0x12345678, 0x87654321]
```

```
    for i, data in enumerate(test_data):
        # Send data using convenience function
        await env['send_data'](data)

        # Write to memory using convenience function
        env['write_memory'](0x1000 + i*4, data)

    # Read back and verify
    for i, expected_data in enumerate(test_data):
        addr = 0x1000 + i*4
        read_data = env['read_memory'](addr)

        # Convert bytearray to integer for comparison
        if isinstance(read_data, bytearray):
            read_value = int.from_bytes(read_data, byteorder='little')
        else:
            read_value = read_data

        assert read_value == expected_data, f"Memory mismatch at
0x{addr:X}"

    # Get comprehensive statistics
    all_stats = env['get_all_stats']()
    log.info(f"Environment test completed: {all_stats}")
```

3.5.4 Advanced Configuration

```
async def create_advanced_gaxi_system(dut, clock):
    """Create advanced GAXI system with custom configuration"""

    # Create custom field configuration
    field_config = FieldConfig()
    field_config.add_field(FieldDefinition("addr", 32, format="hex"))
    field_config.add_field(FieldDefinition("data", 64, format="hex"))
    field_config.add_field(FieldDefinition("cmd", 4, format="hex",
encoding={
        0x0: "NOP", 0x1: "READ", 0x2: "WRITE", 0x3: "BURST"
    }))

    # Create shared memory model
    memory = MemoryModel(
        num_lines=4096,
        bytes_per_line=8,  # 64-bit data
        log=dut._log
    )

    # Create components with advanced configuration
    master = create_gaxi_master(
        dut=dut,
```

```
    title="AdvancedMaster",
    prefix="m_",
    clock=clock,
    field_config=field_config,
    memory_model=memory,
    multi_sig=True,
    mode='fifo_flop',
)

slave = create_gaxi_slave(
    dut=dut,
    title="AdvancedSlave",
    prefix="s_",
    clock=clock,
    field_config=field_config,
    memory_model=memory,
    multi_sig=True,
    mode='fifo_flop',
)

# Create monitors for both sides
master_monitor = create_gaxi_monitor(
    dut=dut,
    title="MasterSideMonitor",
    prefix="m_",
    clock=clock,
    field_config=field_config,
    is_slave=False,
    multi_sig=True
)

slave_monitor = create_gaxi_monitor(
    dut=dut,
    title="SlaveSideMonitor",
    prefix="s_",
    clock=clock,
    field_config=field_config,
    is_slave=True,
    multi_sig=True,
    mode='fifo_flop'
)

# Create scoreboard
scoreboard = create_gaxi_scoreboard(
    name="AdvancedScoreboard",
    field_config=field_config,
    log=dut._log
)
```

```
    return {
        'master': master,
        'slave': slave,
        'master_monitor': master_monitor,
        'slave_monitor': slave_monitor,
        'scoreboard': scoreboard,
        'memory_model': memory,
        'field_config': field_config
    }
```

3.5.5 Multi-Instance Systems

```
async def create_multi_master_system(dut, clock):
    """Create system with multiple masters and one slave"""

    # Shared resources
    shared_memory = MemoryModel(num_lines=2048, bytes_per_line=4)
    shared_config = get_default_field_config(data_width=32)

    # Create multiple masters
    masters = {}
    for i in range(4):
        masters[f'master_{i}'] = create_gaxi_master(
            dut=dut,
            title=f"Master{i}",
            prefix=f"m{i}_",
            clock=clock,
            field_config=shared_config,
            memory_model=shared_memory
        )

    # Create single slave
    slave = create_gaxi_slave(
        dut=dut,
        title="SharedSlave",
        prefix="s_",
        clock=clock,
        field_config=shared_config,
        memory_model=shared_memory
    )

    # Create monitors for each master
    monitors = {}
    for i in range(4):
        monitors[f'master_{i}_monitor'] = create_gaxi_monitor(
            dut=dut,
            title=f"Master{i}Monitor",
            prefix=f"m{i}_",
```

```
            clock=clock,
            field_config=shared_config,
            is_slave=False
        )

    # Slave monitor
    monitors['slave_monitor'] = create_gaxi_monitor(
        dut=dut,
        title="SlaveMonitor",
        prefix="s_",
        clock=clock,
        field_config=shared_config,
        is_slave=True
    )

    # Scoreboard for transaction checking
    scoreboard = create_gaxi_scoreboard(
        name="MultiMasterScoreboard",
        field_config=shared_config
    )

    return {
        'masters': masters,
        'slave': slave,
        'monitors': monitors,
        'scoreboard': scoreboard,
        'shared_memory': shared_memory
    }
```

3.5.6 Factory Pattern Integration

```
class GAXITestFactory:
    """Factory class for creating GAXI test environments"""

    def __init__(self, dut, clock):
        self.dut = dut
        self.clock = clock
        self.default_config = get_default_field_config()

    def create_basic_system(self, prefix=""):
        """Create basic GAXI system"""
        return create_gaxi_system(
            dut=self.dut,
            clock=self.clock,
            title_prefix=prefix
        )

    def create_performance_test_system(self, data_width=32,
memory_size=4096):
```

```
        """Create system optimized for performance testing"""
        return create_gaxi_test_environment(
            dut=self.dut,
            clock=self.clock,
            title_prefix="PERF_",
            data_width=data_width,
            memory_size=memory_size
        )

    def create_protocol_test_system(self, multi_sig=True,
mode='skid'):
        """Create system for protocol compliance testing"""
        field_config = FieldConfig()
        field_config.add_field(FieldDefinition("addr", 32))
        field_config.add_field(FieldDefinition("data", 32))
        field_config.add_field(FieldDefinition("prot", 3))
        field_config.add_field(FieldDefinition("user", 4))

        return create_gaxi_components(
            dut=self.dut,
            clock=self.clock,
            title_prefix="PROT_",
            field_config=field_config,
            multi_sig=multi_sig,
            mode=mode
        )

    def create_stress_test_system(self):
        """Create system for stress testing"""
        return create_gaxi_test_environment(
            dut=self.dut,
            clock=self.clock,
            title_prefix="STRESS_",
            data_width=64,
            memory_size=8192
        )

# Usage
@cocotb.test()
async def test_with_factory(dut):
    factory = GAXITestFactory(dut, dut.clk)

    # Create different systems for different test phases
    basic_system = factory.create_basic_system("BASIC_")
    perf_system = factory.create_performance_test_system(64, 2048)
    stress_system = factory.create_stress_test_system()

    # Run tests with different systems...
```

3.6 Error Handling and Validation

3.6.1 Parameter Validation

```
# Factories automatically handle parameter validation
try:
    master = create_gaxi_master(
        dut=dut,
        title="TestMaster",
        prefix="",
        clock=dut.clk,
        field_config=invalid_config  # Will be validated and corrected
    )
except Exception as e:
    log.error(f"Factory creation failed: {e}")
```

3.6.2 Default Fallbacks

```
# Factories provide sensible defaults for all parameters
master = create_gaxi_master(dut, "Master", "", dut.clk)
# Uses: default field config, dut's logger, empty prefixes, etc.
```

3.6.3 Memory Model Auto-Creation

```
# Memory model is automatically created if not provided
system = create_gaxi_components(dut, dut.clk)
# Automatically creates MemoryModel(1024, 4) with proper configuration
```

3.7 Best Practices

3.7.1 1. Use Empty String Prefixes for Most Cases

```
# Empty prefixes work for most DUT configurations
```

3.7.2 2. Create Complete Systems for Full Testing

```
# Use create_gaxi_test_environment for comprehensive testing
env = create_gaxi_test_environment(dut, dut.clk)
# Includes all components plus convenience functions
```

3.7.3 3. Share Memory Models Across Components

```
# Share memory for consistent state
memory = MemoryModel(1024, 4, log=log)
master = create_gaxi_master(dut, "Master", "", dut.clk,
memory_model=memory)
slave = create_gaxi_slave(dut, "Slave", "", dut.clk,
memory_model=memory)
```

3.7.4 4. Use Multi-Signal Mode for Complex Protocols

```
# Enable multi-signal mode for protocols with many fields
components = create_gaxi_components(dut, dut.clk, multi_sig=True)
```

3.7.5 5. Leverage Factory Patterns for Complex Tests

```
# Create factory class for standardized test setups
factory = GAXITestFactory(dut, dut.clk)
system = factory.create_protocol_test_system()
```

3.8 Backward Compatibility

All factory functions maintain complete backward compatibility:

- **All existing parameters are preserved** and passed through exactly as before
- **Parameter order is maintained** for positional arguments
- **Default values are enhanced** but don't break existing code
- **New parameters have sensible defaults** that work with existing configurations
- **Legacy parameter names are supported** (e.g., `field_mode`, `optional_signal_map`) even when unused; `signal_map` is forwarded

```
# Legacy code continues to work unchanged
components = create_gaxi_components(
    dut, clock, title_prefix="Legacy_", field_mode=True,
    signal_map=old_signal_map, optional_signal_map=old_optional_map
)
```

The GAXI factories provide a robust, simplified interface for creating GAXI components while maintaining full backward compatibility and supporting advanced configuration scenarios.

4 gaxi_master.py

GAXI Master with integrated structured pipeline that maintains all existing functionality and timing while adding better debugging and error recovery through structured pipeline phases.

4.1 Overview

The `GAXIMaster` class provides: - **Structured pipeline** with enhanced debugging and error recovery - **Exact timing compatibility** with existing code - **Signal resolution** and data driving from GAXIComponentBase - **Memory model integration** using base MemoryModel directly - **Comprehensive statistics** including pipeline performance - **Optional pipeline debugging** for troubleshooting

Inherits common functionality from GAXIComponentBase and extends CocoTB BusDriver.

4.2 Class

4.2.1 GAXIMaster

```
class GAXIMaster(GAXIComponentBase, BusDriver):
    def __init__(self, dut, title, prefix, clock, field_config,
                 timeout_cycles=1000, mode='skid', bus_name='',
pkt_prefix='',
                 multi_sig=False, randomizer=None, memory_model=None,
                 log=None, super_debug=False, pipeline_debug=False,
                 signal_map=None, protocol_type='gaxi_master',
**kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title/name - `prefix`: Bus prefix - `clock`: Clock signal - `field_config`: Field configuration - `timeout_cycles`: Timeout for handshake operations (default: 1000) - `mode`: GAXI mode ('skid', 'fifo_mux', 'fifo_flop') - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `multi_sig`: Whether using multi-signal mode - `randomizer`: Optional randomizer for timing - `memory_model`: Optional memory model for transactions - `log`: Logger instance (required — raises ValueError if None; pass your TBBase logger) - `super_debug`: Enable detailed debugging - `pipeline_debug`: Enable pipeline phase debugging - `signal_map`: Optional manual signal mapping override - `**kwargs`: Additional arguments

4.3 Core Methods

4.3.1 Transaction Sending

4.3.1.1 async send(packet)

Send a packet and wait for completion.

Parameters: - `packet`: Packet to send

Returns: True when complete

```
# Create and send packet
packet = master.create_packet(addr=0x1000, data=0xDEADBEEF)
await master.send(packet)
```

4.3.1.2 async busy_send(transaction)

Send a transaction and wait for completion with busy checking.

Parameters: - `transaction`: Transaction to send

```
# Send with busy checking
transaction = master.create_packet(data=0x12345678)
await master.busy_send(transaction)
```

4.3.1.3 *create_packet(**field_values)*

Create a packet with specified field values.

Parameters: - `**field_values`: Initial field values

Returns: GAXIPacket instance

```
# Create packet with fields
packet = master.create_packet(
    addr=0x2000,
    data=0xCAFEBABE,
    cmd=0x1  # READ command
)
```

4.3.2 Bus Management

4.3.2.1 *async reset_bus()*

Reset bus with enhanced pipeline state management.

```
await master.reset_bus()
```

4.3.3 Memory Operations

4.3.3.1 *async write_to_memory(packet)*

Write packet to memory using unified memory integration.

Parameters: - `packet`: Packet to write

Returns: Success boolean

```
packet = master.create_packet(addr=0x1000, data=0xDEADBEEF)
success = await master.write_to_memory(packet)
if not success:
    log.error("Memory write failed")
```

4.3.3.2 *async read_from_memory(packet)*

Read data from memory using unified memory integration.

Parameters: - `packet`: Packet with address to read

Returns: Tuple of (success, data)

```
packet = master.create_packet(addr=0x1000)
success, data = await master.read_from_memory(packet)
```

```
if success:
    log.info(f"Read data: 0x{data:X}")
```

4.3.4 Pipeline Control and Debugging

4.3.4.1 enable_pipeline_debug(enable=True)

Enable or disable pipeline debugging at runtime.

Parameters: - `enable`: Enable debugging flag

```
# Enable detailed pipeline logging
master.enable_pipeline_debug(True)

# Disable for performance
master.enable_pipeline_debug(False)
```

4.3.4.2 get_pipeline_stats()

Get pipeline-specific statistics.

Returns: Dictionary with pipeline statistics

```
pipeline_stats = master.get_pipeline_stats()
print(f"Current state: {pipeline_stats['current_state']}")
print(f"Queue depth: {pipeline_stats['queue_depth']}")
print(f"Phase counts: P1={pipeline_stats['phase1_count']}")
```

4.3.4.3 get_pipeline_debug_info()

Get detailed pipeline debug information.

Returns: Dictionary with debug information

```
debug_info = master.get_pipeline_debug_info()
print(f"Phase timings: {debug_info['phase_timings']}")
print(f"Statistics: {debug_info['phase_statistics']}")
```

4.3.5 Statistics

4.3.5.1 get_stats()

Get comprehensive statistics including pipeline stats.

Returns: Dictionary containing all statistics

```
stats = master.get_stats()
print(f"Master stats: {stats['master_stats']}")
print(f"Base stats: {stats}")
print(f"Pipeline stats: {stats['pipeline_stats']}")
```

4.4 Pipeline Architecture

The GAXIMaster uses a structured 3-phase pipeline:

4.4.1 Phase 1: Apply Delays

- Apply `valid_delay` from randomizer
- Deassert valid and clear data during delay
- Maintains exact original timing

4.4.2 Phase 2: Drive and Handshake

- Drive signals for transaction
- Assert valid signal
- Wait for ready handshake (with timeout)
- Enhanced error recovery on timeout

4.4.3 Phase 3: Complete Transfer

- Capture completion time
- Deassert valid
- Clear data bus
- Move to sent queue

4.4.4 Pipeline State Tracking

```
# Pipeline states
"idle" → "queued" → "pipeline_active" → "transaction_start" →
"phase1" → "phase2" → "phase3" → "transaction_complete" → "idle"

# Error states
"error_recovery", "reset"
```

4.5 Usage Patterns

4.5.1 Basic Usage

```
import cocotb
from cocotb.triggers import RisingEdge
from CocoTBFramework.components.gaxi import GAXIMaster
from CocoTBFramework.components.shared.field_config import FieldConfig

@cocotb.test()
async def test_gaxi_master(dut):
    clock = dut.clk
```

```
    # Create field configuration
    field_config = FieldConfig()
    field_config.add_field(FieldDefinition("addr", 32, format="hex"))
    field_config.add_field(FieldDefinition("data", 32, format="hex"))

    # Create master
    master = GAXIMaster(
        dut=dut,
        title="TestMaster",
        prefix="",
        clock=clock,
        field_config=field_config,
        log=log,              # Required
        pipeline_debug=True  # Enable pipeline debugging
    )

    # Reset bus
    await master.reset_bus()

    # Send transactions
    for i in range(10):
        packet = master.create_packet(
            addr=0x1000 + i*4,
            data=0x12340000 + i
        )
        await master.send(packet)

    # Get statistics
    stats = master.get_stats()
    print(f"Sent {stats['master_stats']['transactions_completed']}
transactions")
```

4.5.2 Advanced Configuration

```
from CocoTBFramework.components.shared.flex_randomizer import
FlexRandomizer
from CocoTBFramework.components.shared.memory_model import MemoryModel

# Create randomizer for timing
randomizer = FlexRandomizer({
    'valid_delay': ([(0, 0), (1, 5), (10, 20)], [0.6, 0.3, 0.1])
})

# Create memory model
memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)

# Create master with advanced configuration
master = GAXIMaster(
```

```
    dut=dut,
    title="AdvancedMaster",
    prefix="m_",
    clock=clock,
    field_config=field_config,
    randomizer=randomizer,
    memory_model=memory,
    timeout_cycles=5000,
    pipeline_debug=True,
    multi_sig=True
)
```

4.5.3 Memory-Integrated Testing

```
@cocotb.test()
async def test_memory_operations(dut):
    master = GAXIMaster(dut, "MemMaster", "", clock, field_config,
                        memory_model=memory)

    # Write to memory
    write_packet = master.create_packet(addr=0x1000, data=0xDEADBEEF)
    success = await master.write_to_memory(write_packet)
    assert success, "Memory write failed"

    # Read from memory
    read_packet = master.create_packet(addr=0x1000)
    success, data = await master.read_from_memory(read_packet)
    assert success, "Memory read failed"
    assert data == 0xDEADBEEF, f"Data mismatch: expected 0xDEADBEEF,
got 0x{data:X}"

    # Send read transaction
    await master.send(read_packet)
```

4.5.4 Pipeline Debugging

```
@cocotb.test()
async def test_with_pipeline_debug(dut):
    master = GAXIMaster(dut, "DebugMaster", "", clock, field_config,
                        pipeline_debug=True)

    # Send transaction with detailed logging
    packet = master.create_packet(addr=0x1000, data=0x12345678)
    await master.send(packet)

    # Analyze pipeline performance
    pipeline_stats = master.get_pipeline_stats()
    print(f"Pipeline Statistics:")
    print(f"  Phase 1 count: {pipeline_stats['phase1_count']}")
    print(f"  Phase 2 count: {pipeline_stats['phase2_count']}")
```

```
    print(f"  Phase 3 count: {pipeline_stats['phase3_count']}")
    print(f"  Timeouts: {pipeline_stats['timeout_count']}")
    print(f"  Errors: {pipeline_stats['error_count']}")

    # Get debug info
    debug_info = master.get_pipeline_debug_info()
    for state, timing in debug_info['phase_timings'].items():
        print(f"  {state}: {timing}ns")
```

4.5.5 Burst Transactions

```
async def send_burst_transactions(master, base_addr, count):
    """Send a burst of transactions"""
    for i in range(count):
        packet = master.create_packet(
            addr=base_addr + i*4,
            data=0x10000000 + i
        )
        await master.send(packet)

    # Get performance metrics
    stats = master.get_stats()
    master_stats = stats['master_stats']

    print(f"Burst complete:")
    print(f"  Transactions: {master_stats['transactions_completed']}")
    print(f"  Average latency:
{master_stats['average_latency_ms']:.2f}ms")
    print(f"  Throughput: {master_stats['current_throughput_tps']:.1f}
TPS")
```

4.5.6 Error Handling and Recovery

```
@cocotb.test()
async def test_error_recovery(dut):
    master = GAXIMaster(dut, "ErrorMaster", "", clock, field_config,
                        timeout_cycles=100,  # Short timeout for
testing
                        pipeline_debug=True)

    try:
        # This might timeout if slave is not ready
        packet = master.create_packet(addr=0x1000, data=0x12345678)
        await master.send(packet)

    except Exception as e:
        log.error(f"Transaction failed: {e}")

        # Check pipeline state
        debug_info = master.get_pipeline_debug_info()
```

```
        print(f"Pipeline state: {debug_info['current_state']}")

        # Reset and try again
        await master.reset_bus()
        await master.send(packet)  # Should work after reset
```

4.5.7 Performance Monitoring

```
async def monitor_performance(master, duration_ms=1000):
    """Monitor master performance over time"""
    import asyncio

    start_time = get_sim_time('ns')
    end_time = start_time + duration_ms * 1e6  # Convert to ns

    while get_sim_time('ns') < end_time:
        # Send transaction
        packet = master.create_packet(
            addr=random.randint(0x1000, 0x8000),
            data=random.randint(0, 0xFFFFFFFF)
        )
        await master.send(packet)

        # Brief delay
        await asyncio.sleep(0.001)  # 1ms

    # Analyze performance
    stats = master.get_stats()
    master_stats = stats['master_stats']
    pipeline_stats = stats['pipeline_stats']

    print(f"Performance Analysis ({duration_ms}ms):")
    print(f"  Transactions sent: {master_stats['transactions_sent']}")
    print(f"  Success rate: {master_stats['success_rate_percent']:.1f}
%")
    print(f"  Average latency:
{master_stats['average_latency_ms']:.2f}ms")
    print(f"  Peak throughput:
{master_stats['peak_throughput_tps']:.1f} TPS")
    print(f"  Pipeline errors: {pipeline_stats['error_count']}")
```

4.6 Error Handling

4.6.1 Timeout Handling

```
# Timeouts are handled automatically with proper cleanup
try:
    await master.send(packet)
except TimeoutError:
```

```
    # Pipeline automatically recovers
    print("Transaction timed out but pipeline recovered")
```

4.6.2 Signal Mapping Errors

```
try:
    master = GAXIMaster(dut, "Master", "", clock, field_config)
except RuntimeError as e:
    # Try manual signal mapping
    signal_map = {'valid': 'custom_valid', 'ready': 'custom_ready'}
    master = GAXIMaster(dut, "Master", "", clock, field_config,
                        signal_map=signal_map)
```

4.6.3 Memory Operation Errors

```
success = await master.write_to_memory(packet)
if not success:
    # Memory operation failed
    stats = master.get_stats()
    if 'memory_stats' in stats:
        print(f"Memory errors: {stats['memory_stats']
['boundary_violations']}")
```

4.7 Best Practices

4.7.1 1. Enable Pipeline Debugging During Development

```
master = GAXIMaster(..., pipeline_debug=True)  # Development
master = GAXIMaster(..., pipeline_debug=False) # Production
```

4.7.2 2. Use Appropriate Timeout Values

```
# Short timeout for fast testing
master = GAXIMaster(..., timeout_cycles=100)

# Long timeout for realistic conditions
master = GAXIMaster(..., timeout_cycles=10000)
```

4.7.3 3. Monitor Statistics Regularly

```
# Check statistics periodically
if transaction_count % 100 == 0:
    stats = master.get_stats()
    check_performance_requirements(stats)
```

4.7.4 4. Handle Reset Properly

```
# Reset before starting transactions
await master.reset_bus()

# Reset on error recovery
```

```
if error_detected:
    await master.reset_bus()
```

4.7.5 5. Use Memory Model for Complex Testing

```
# Create memory model for transaction validation
memory = MemoryModel(1024, 4, log=log)
master = GAXIMaster(..., memory_model=memory)

# Validate transactions through memory
await master.write_to_memory(packet)
success, data = await master.read_from_memory(packet)
```

The GAXIMaster provides a robust, high-performance foundation for GAXI transaction generation with comprehensive debugging, error recovery, and performance monitoring capabilities.

5 gaxi_monitor.py

GAXI Monitor implementation using unified base functionality. Preserves exact timing-critical cocotb methods and external API while eliminating code duplication through inheritance from unified base classes.

5.1 Overview

The `GAXIMonitor` class provides: - **Pure monitoring functionality** with no signal driving - **Master-side or slave-side monitoring** based on is_slave parameter - **Protocol violation detection** and reporting - **Handshake tracking** with precise timing - **Inherited functionality** from GAXIMonitorBase for common operations - **Mode-specific data sampling** with corrected timing

Inherits all common functionality from GAXIMonitorBase including signal resolution, data collection, packet management, and memory integration.

5.2 Class

5.2.1 GAXIMonitor

```
class GAXIMonitor(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
is_slave=False,
                 mode='skid', bus_name='', pkt_prefix='',
multi_sig=False,
                 log=None, super_debug=False, signal_map=None,
                 protocol_type=None, **kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title/name - `prefix`: Bus prefix - `clock`: Clock signal - `field_config`: Field configuration - `is_slave`: If True, monitor slave side; if False,

monitor master side - `mode`: GAXI mode ('skid', 'fifo_mux', 'fifo_flop') - for DUT parameter only - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `multi_sig`: Whether using multi-signal mode - `log`: Logger instance (required — raises ValueError if None; pass your TBBase logger) - `super_debug`: Enable detailed debugging - `signal_map`: Optional manual signal mapping override - `**kwargs`: Additional arguments

5.3 Data Sampling Rules

The GAXIMonitor uses corrected sampling timing based on mode and monitoring side:

5.3.1 Master Side Monitoring (`is_slave=False`)

- **All modes**: Always sample immediately when handshake detected
- **Rationale**: Master side signals are stable at handshake time
- **Performance**: Lowest latency monitoring

5.3.2 Slave Side Monitoring (`is_slave=True`)

- **Skid mode**: Sample immediately when handshake detected
- **FIFO MUX mode**: Sample immediately when handshake detected

- **FIFO FLOP mode**: Delay capture by one cycle after handshake
- **Rationale**: Matches DUT internal timing for different implementations

5.4 Core Methods

5.4.1 Inherited Methods

GAXIMonitor inherits all methods from GAXIMonitorBase: - `create_packet(**field_values)` - Create packets - `get_observed_packets(count=None)` - Get observed transactions - `clear_queue()` - Clear observation queue - `handle_memory_write(packet)` - Memory integration - `handle_memory_read(packet)` - Memory integration - `get_base_stats()` - Statistics collection

5.4.2 Monitor-Specific Methods

5.4.2.1 `get_stats()`

Get comprehensive statistics including monitor-specific information.

Returns: Dictionary containing all statistics

```
stats = monitor.get_stats()
print(f"Monitor type: {stats['monitor_type']}")  # 'master' or 'slave'
print(f"Observed packets: {stats['observed_packets']}")
```

```
print(f"Protocol violations: {stats['monitor_stats']
['protocol_violations']}")
```

5.5 Usage Patterns

5.5.1 Basic Master Side Monitoring

```
import cocotb
from cocotb.triggers import Timer
from CocoTBFramework.components.gaxi import GAXIMonitor
from CocoTBFramework.components.shared.field_config import FieldConfig

@cocotb.test()
async def test_master_monitoring(dut):
    clock = dut.clk

    # Create field configuration
    field_config = FieldConfig()
    field_config.add_field(FieldDefinition("addr", 32, format="hex"))
    field_config.add_field(FieldDefinition("data", 32, format="hex"))

    # Create master side monitor
    master_monitor = GAXIMonitor(
        dut=dut,
        title="MasterMonitor",
        prefix="m_",  # Monitor master interface
        clock=clock,
        field_config=field_config,
        is_slave=False,  # Monitor master side
        mode='skid',
        log=log  # Required
    )

    # Add callback to process observed transactions
    def log_transaction(packet):
        print(f"Master sent: addr=0x{packet.addr:X},
data=0x{packet.data:X}")

    master_monitor.add_callback(log_transaction)

    # Run test and let monitor observe
    await Timer(1000, units='ns')

    # Check observed transactions
    packets = master_monitor.get_observed_packets()
    print(f"Master monitor observed {len(packets)} transactions")
```

5.5.2 Basic Slave Side Monitoring

```
@cocotb.test()
async def test_slave_monitoring(dut):
    clock = dut.clk

    # Create field configuration
    field_config = FieldConfig()
    field_config.add_field(FieldDefinition("addr", 32, format="hex"))
    field_config.add_field(FieldDefinition("data", 32, format="hex"))

    # Create slave side monitor with FIFO FLOP mode
    slave_monitor = GAXIMonitor(
        dut=dut,
        title="SlaveMonitor",
        prefix="s_",  # Monitor slave interface
        clock=clock,
        field_config=field_config,
        is_slave=True,   # Monitor slave side
        mode='fifo_flop' # DUT uses registered interface
    )

    # Add callback to process observed transactions
    def process_slave_transaction(packet):
        print(f"Slave received: addr=0x{packet.addr:X},
data=0x{packet.data:X}")

        # Additional processing
        if packet.addr >= 0x8000:
            print("  → High address region")

    slave_monitor.add_callback(process_slave_transaction)

    # Run test
    await Timer(1000, units='ns')

    # Analyze slave activity
    packets = slave_monitor.get_observed_packets()
    print(f"Slave monitor observed {len(packets)} transactions")
```

5.5.3 Dual Monitor Setup

```
@cocotb.test()
async def test_dual_monitoring(dut):
    clock = dut.clk
    field_config = create_field_config()

    # Monitor both sides of the interface
    master_monitor = GAXIMonitor(
```

```
        dut=dut,
        title="MasterSideMonitor",
        prefix="m_",
        clock=clock,
        field_config=field_config,
        is_slave=False  # Master side
    )

    slave_monitor = GAXIMonitor(
        dut=dut,
        title="SlaveSideMonitor",
        prefix="s_",
        clock=clock,
        field_config=field_config,
        is_slave=True   # Slave side
    )

    # Track transactions on both sides
    master_transactions = []
    slave_transactions = []

    def track_master(packet):
        master_transactions.append(packet)
        print(f"Master → Slave: {packet.formatted(compact=True)}")

    def track_slave(packet):
        slave_transactions.append(packet)
        print(f"Slave ← Master: {packet.formatted(compact=True)}")

    master_monitor.add_callback(track_master)
    slave_monitor.add_callback(track_slave)

    # Run test
    await Timer(2000, units='ns')

    # Compare transaction counts
    print(f"Master side: {len(master_transactions)} transactions")
    print(f"Slave side: {len(slave_transactions)} transactions")

    # Verify transaction matching
    assert len(master_transactions) == len(slave_transactions), \
        "Transaction count mismatch between master and slave sides"
```

5.5.4 Protocol Violation Monitoring

```
@cocotb.test()
async def test_protocol_violations(dut):
    # Create monitor with protocol checking
    monitor = GAXIMonitor(
```

```
    dut=dut,
    title="ProtocolMonitor",
    prefix="",
    clock=clock,
    field_config=field_config,
    is_slave=False,
    super_debug=True  # Enable detailed logging
)

# Track protocol violations
violations = []

def check_protocol(packet):
    """Check for protocol violations in observed transactions"""
    # Check address alignment
    if hasattr(packet, 'addr') and packet.addr % 4 != 0:
        violation = f"Unaligned address: 0x{packet.addr:X}"
        violations.append(violation)
        print(f"VIOLATION: {violation}")

    # Check for X/Z values
    for field_name in ['addr', 'data']:
        if hasattr(packet, field_name):
            value = getattr(packet, field_name)
            if value == -1:  # X/Z represented as -1
                violation = f"X/Z value in {field_name}"
                violations.append(violation)
                print(f"VIOLATION: {violation}")

monitor.add_callback(check_protocol)

# Run test
await Timer(1000, units='ns')

# Report violations
if violations:
    print(f"Detected {len(violations)} protocol violations:")
    for violation in violations:
        print(f"  - {violation}")
else:
    print("No protocol violations detected")

# Get monitor statistics
stats = monitor.get_stats()
print(f"Monitor statistics: {stats['monitor_stats']}")
```

5.5.5 Performance Monitoring

```
@cocotb.test()
async def test_performance_monitoring(dut):
    monitor = GAXIMonitor(dut, "PerfMonitor", "", clock, field_config)

    # Performance tracking
    transaction_times = []
    inter_transaction_times = []
    last_time = 0

    def track_performance(packet):
        current_time = get_sim_time('ns')
        transaction_times.append(current_time)

        if last_time > 0:
            inter_time = current_time - last_time
            inter_transaction_times.append(inter_time)

        nonlocal last_time
        last_time = current_time

    monitor.add_callback(track_performance)

    # Run test
    await Timer(5000, units='ns')

    # Analyze performance
    if inter_transaction_times:
        avg_interval = sum(inter_transaction_times) /
len(inter_transaction_times)
        throughput = 1e9 / avg_interval if avg_interval > 0 else 0

        print(f"Performance Analysis:")
        print(f"  Transactions: {len(transaction_times)}")
        print(f"  Average interval: {avg_interval:.2f} ns")
        print(f"  Throughput: {throughput:.1f} TPS")
        print(f"  Min interval: {min(inter_transaction_times):.2f}
ns")
        print(f"  Max interval: {max(inter_transaction_times):.2f}
ns")
```

5.5.6 Mode-Specific Configuration

```
def create_mode_specific_monitors(dut, clock, field_config):
    """Create monitors for different DUT modes"""

    monitors = {}
```

```
    # Skid mode monitor (immediate capture)
    monitors['skid'] = GAXIMonitor(
        dut=dut,
        title="SkidMonitor",
        prefix="skid_",
        clock=clock,
        field_config=field_config,
        is_slave=True,
        mode='skid'
    )

    # FIFO MUX mode monitor (immediate capture)
    monitors['fifo_mux'] = GAXIMonitor(
        dut=dut,
        title="FifoMuxMonitor",
        prefix="mux_",
        clock=clock,
        field_config=field_config,
        is_slave=True,
        mode='fifo_mux'
    )

    # FIFO FLOP mode monitor (delayed capture)
    monitors['fifo_flop'] = GAXIMonitor(
        dut=dut,
        title="FifoFlopMonitor",
        prefix="flop_",
        clock=clock,
        field_config=field_config,
        is_slave=True,
        mode='fifo_flop'  # One cycle delay for data capture
    )

    return monitors

@cocotb.test()
async def test_mode_specific_monitoring(dut):
    monitors = create_mode_specific_monitors(dut, clock, field_config)

    # Add same callback to all monitors
    def log_transaction(packet):
        print(f"Transaction: {packet.formatted(compact=True)}")

    for mode, monitor in monitors.items():
        monitor.add_callback(log_transaction)

    # Run test
    await Timer(1000, units='ns')
```

```
    # Compare results across modes
    for mode, monitor in monitors.items():
        packets = monitor.get_observed_packets()
        stats = monitor.get_stats()
        print(f"{mode} mode: {len(packets)} packets,
{stats['monitor_type']} side")
```

5.5.7 Integration with Scoreboards

```
from CocoTBFramework.scoreboards.gaxi_scoreboard import GAXIScoreboard

@cocotb.test()
async def test_scoreboard_integration(dut):
    # Create scoreboard
    scoreboard = GAXIScoreboard("TestScoreboard", field_config,
log=log)

    # Create monitors
    master_monitor = GAXIMonitor(dut, "MasterMon", "m_", clock,
field_config,
                                 is_slave=False)
    slave_monitor = GAXIMonitor(dut, "SlaveMon", "s_", clock,
field_config,
                                is_slave=True)

    # Connect monitors to scoreboard
    master_monitor.add_callback(scoreboard.add_expected)
    slave_monitor.add_callback(scoreboard.add_actual)

    # Run test
    await Timer(2000, units='ns')

    # Check scoreboard results
    error_count = scoreboard.report()
    print(f"Scoreboard: {scoreboard.transaction_count} transactions
compared, "
          f"{error_count} errors")

    # Verify all transactions matched
    assert error_count == 0, "Transaction mismatches detected"
```

5.5.8 Memory Validation Monitoring

```
@cocotb.test()
async def test_memory_validation(dut):
    # Create memory model for validation
    memory = MemoryModel(num_lines=256, bytes_per_line=4, log=log)
```

```
    # Create monitor with memory integration
    monitor = GAXIMonitor(
        dut=dut,
        title="MemoryMonitor",
        prefix="",
        clock=clock,
        field_config=field_config,
        memory_model=memory,
        is_slave=True
    )

    # Track memory operations
    def validate_memory_transaction(packet):
        """Validate transactions against memory model"""
        if hasattr(packet, 'cmd'):
            if packet.cmd == 2:  # Write
                success = monitor.handle_memory_write(packet)
                if success:
                    print(f"Memory write: addr=0x{packet.addr:X}, "
                          f"data=0x{packet.data:X}")
                else:
                    print(f"Memory write failed:
addr=0x{packet.addr:X}")

            elif packet.cmd == 1:  # Read
                success, data = monitor.handle_memory_read(packet)
                if success:
                    print(f"Memory read: addr=0x{packet.addr:X}, "
                          f"data=0x{data:X}")
                    # Validate read data if available
                    if hasattr(packet, 'data') and packet.data !=
data:
                        print(f"  WARNING: Expected 0x{data:X}, "
                              f"got 0x{packet.data:X}")

    monitor.add_callback(validate_memory_transaction)

    # Run test
    await Timer(1000, units='ns')

    # Get memory statistics
    stats = monitor.get_stats()
    if 'memory_stats' in stats:
        memory_stats = stats['memory_stats']
        print(f"Memory operations: reads={memory_stats['reads']}, "
              f"writes={memory_stats['writes']}")
```

5.6 Advanced Features

5.6.1 Custom Signal Mapping

```
# For non-standard signal names
signal_map = {
    'valid': 'master_transaction_valid',
    'ready': 'slave_ready_signal',
    'data': 'transaction_data_bus'
}

monitor = GAXIMonitor(
    dut=dut,
    title="CustomMonitor",
    prefix="",
    clock=clock,
    field_config=field_config,
    signal_map=signal_map  # Override automatic discovery
)
```

5.6.2 Multi-Field Monitoring

```
# Create field configuration with multiple fields
field_config = FieldConfig()
field_config.add_field(FieldDefinition("addr", 32, format="hex"))
field_config.add_field(FieldDefinition("data", 32, format="hex"))
field_config.add_field(FieldDefinition("cmd", 4, format="hex"))
field_config.add_field(FieldDefinition("id", 8, format="hex"))

# Monitor with multi-signal mode
monitor = GAXIMonitor(
    dut=dut,
    title="MultiFieldMonitor",
    prefix="",
    clock=clock,
    field_config=field_config,
    multi_sig=True  # Individual signals for each field
)
```

5.7 Error Handling

5.7.1 Signal Resolution Errors

```
try:
    monitor = GAXIMonitor(dut, "Monitor", "", clock, field_config)
except RuntimeError as e:
    print(f"Signal resolution failed: {e}")
    # Try with manual signal mapping
```

```
    signal_map = create_manual_signal_map()
    monitor = GAXIMonitor(dut, "Monitor", "", clock, field_config,
                          signal_map=signal_map)
```

5.7.2 Monitoring Errors

```
# Monitor automatically handles signal errors
# Check statistics for error information
stats = monitor.get_stats()
monitor_stats = stats['monitor_stats']
if monitor_stats['x_z_violations'] > 0:
    print(f"X/Z violations detected:
{monitor_stats['x_z_violations']}")
```

5.8 Best Practices

5.8.1 1. Choose Appropriate Monitoring Side

```
# Monitor master side to see outgoing transactions
master_monitor = GAXIMonitor(..., is_slave=False)

# Monitor slave side to see received transactions
slave_monitor = GAXIMonitor(..., is_slave=True)
```

5.8.2 2. Match Mode to DUT Implementation

```
# For DUT with registered slave interface
monitor = GAXIMonitor(..., is_slave=True, mode='fifo_flop')

# For DUT with combinational interface
monitor = GAXIMonitor(..., is_slave=True, mode='skid')
```

5.8.3 3. Use Callbacks for Real-Time Processing

```
# Prefer callbacks over polling
monitor.add_callback(process_transaction)

# Avoid polling the queue directly
# while True:
#     if monitor._recvQ:  # Don't do this
packets = monitor.get_observed_packets()  # Do this instead
```

5.8.4 4. Enable Debug During Development

```
monitor = GAXIMonitor(..., super_debug=True)   # Development
monitor = GAXIMonitor(..., super_debug=False)  # Production
```

5.8.5 5. Monitor Statistics for Health

```
# Regular statistics checking
stats = monitor.get_stats()
```

```
if stats['monitor_stats']['protocol_violations'] > 0:
    print("Protocol violations detected - investigate")
```

The GAXIMonitor provides comprehensive transaction observation capabilities with precise timing control, protocol violation detection, and flexible configuration for thorough verification of GAXI protocol implementations.

6 gaxi_monitor_base.py

Base class providing common GAXI monitoring functionality using unified infrastructure. Eliminates duplication while preserving exact APIs and timing-critical behavior.

6.1 Overview

The `GAXIMonitorBase` class provides: - **Common monitoring functionality** shared by GAXIMonitor and GAXISlave - **Signal resolution and data collection** setup from GAXIComponentBase - **Unified field configuration** handling and packet management - **Memory model integration** using base MemoryModel directly - **Clean data collection** with automatic field unpacking - **Unified packet finishing** without conditional complexity

This base class eliminates code duplication between monitor components while preserving exact timing-critical behavior.

6.2 Class

6.2.1 GAXIMonitorBase

```
class GAXIMonitorBase(GAXIComponentBase, BusMonitor):
    def __init__(self, dut, title, prefix, clock, field_config,
                 mode='skid', bus_name='', pkt_prefix='',
multi_sig=False,
                 protocol_type=None,  # Set by subclass
                 log=None, super_debug=False, signal_map=None,
**kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title/name - `prefix`: Bus prefix - `clock`: Clock signal - `field_config`: Field configuration - `mode`: GAXI mode (‘skid’, ‘fifo_mux’, ‘fifo_flop’) - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `multi_sig`: Whether using multi-signal mode - `protocol_type`: Must be set by subclass (‘gaxi_master’ or ‘gaxi_slave’) - `log`: Logger instance (required — raises ValueError if None; pass your TBBase logger) - `super_debug`: Enable detailed debugging - `signal_map`: Optional manual signal mapping override - `**kwargs`: Additional arguments for BusMonitor

6.3 Core Methods

6.3.1 Data Collection

6.3.1.1 _get_data_dict()

Clean data collection with automatic field unpacking.

This replaces messy conditional unpacking logic that was duplicated in both GAXIMonitor and GAXISlave. Uses the unified DataCollectionStrategy.collect_and_unpack_data() method.

Returns: Dictionary of field values, properly unpacked

```
# Internal method used by subclasses
data_dict = self._get_data_dict()
# Returns: {'addr': 0x1000, 'data': 0xDEADBEEF, 'cmd': 0x2}
```

6.3.1.2 _finish_packet(current_time, packet, data_dict=None)

Clean packet finishing without conditional complexity.

Replaces duplicate _finish_packet logic that was in both GAXIMonitor and GAXISlave with identical functionality.

Parameters: - `current_time`: Current simulation time - `packet`: Packet to finish - `data_dict`: Optional field data (if None, will collect fresh data)

```
# Internal method used by subclasses
packet = GAXIPacket(self.field_config)
packet.start_time = current_time
data_dict = self._get_data_dict()
self._finish_packet(current_time, packet, data_dict)
```

6.3.2 Packet Management

*6.3.2.1 create_packet(**field_values)*

Create a packet with specified field values.

Parameters: - `**field_values`: Initial field values

Returns: GAXIPacket instance with specified field values

```
# Create packet with initial values
packet = monitor.create_packet(addr=0x1000, data=0xDEADBEEF)
```

6.3.2.2 get_observed_packets(count=None)

Get observed packets from standard cocotb _recvQ.

Parameters: - `count`: Number of packets to return (None = all)

Returns: List of observed packets

```
# Get all observed packets
all_packets = monitor.get_observed_packets()

# Get last 10 packets
recent_packets = monitor.get_observed_packets(10)
```

6.3.2.3 `clear_queue()`

Clear the observed transactions queue using standard cocotb pattern.

```
monitor.clear_queue()
```

6.3.3 Memory Operations

6.3.3.1 `handle_memory_write(packet)`

Handle memory write using unified memory integration.

Parameters: - `packet`: Packet to write to memory

Returns: Success boolean

```
success = monitor.handle_memory_write(packet)
if success:
    log.debug("Memory write successful")
```

6.3.3.2 `handle_memory_read(packet)`

Handle memory read using unified memory integration.

Parameters: - `packet`: Packet with address to read

Returns: Tuple of (success, data)

```
success, data = monitor.handle_memory_read(packet)
if success:
    log.debug(f"Memory read successful, data=0x{data:X}")
```

6.3.4 Statistics

6.3.4.1 `get_base_stats()`

Get base statistics common to all GAXI monitoring components.

Subclasses should call this and add their own specific statistics.

Returns: Dictionary containing base statistics

```
base_stats = monitor.get_base_stats()
print(f"Monitor stats: {base_stats['monitor_stats']}")
print(f"Observed packets: {base_stats['observed_packets']}")
```

6.4 Internal Architecture

6.4.1 Data Flow

```
Signal Changes → _get_data_dict() → Packet Creation →
_finish_packet() → _recv() → Callbacks → Statistics Update
```

6.4.2 Packet Processing Pipeline

1. **Signal sampling** using unified DataCollectionStrategy
2. **Data unpacking** with automatic field handling
3. **Packet creation** with timing information
4. **Field population** using packet unpack methods
5. **Statistics update** and callback triggering
6. **Queue management** using standard cocotb patterns

6.4.3 Memory Integration

- Automatic memory writes for received transactions
- Memory read support for validation
- Error handling and logging
- Statistics collection for memory operations

6.5 Usage Patterns

6.5.1 Creating a Custom Monitor

```
from CocoTBFramework.components.gaxi.gaxi_monitor_base import
GAXIMonitorBase
from CocoTBFramework.components.gaxi.gaxi_packet import GAXIPacket

class CustomGAXIMonitor(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
                 monitor_master_side=True, **kwargs):
        # Determine protocol type based on what we're monitoring
        protocol_type = 'gaxi_master' if monitor_master_side else
'gaxi_slave'

        super().__init__(
            dut=dut,
```

```
            title=title,
            prefix=prefix,
            clock=clock,
            field_config=field_config,
            protocol_type=protocol_type,
            **kwargs
        )

        self.monitor_master_side = monitor_master_side

    async def _monitor_recv(self):
        """Custom monitoring implementation"""
        while True:
            await FallingEdge(self.clock)

            # Check for handshake
            if (hasattr(self, 'valid_sig') and hasattr(self,
'ready_sig') and
                self.valid_sig.value == 1 and self.ready_sig.value ==
1):

                # Create packet
                packet = GAXIPacket(self.field_config)
                packet.start_time = get_sim_time('ns')

                # Use unified data collection
                data_dict = self._get_data_dict()

                # Use unified packet finishing
                self._finish_packet(get_sim_time('ns'), packet,
data_dict)
```

6.5.2 Memory-Integrated Monitor

```
class MemoryValidatingMonitor(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
                 memory_model, **kwargs):
        super().__init__(
            dut=dut,
            title=title,
            prefix=prefix,
            clock=clock,
            field_config=field_config,
            protocol_type='gaxi_slave',  # Monitor slave side
            memory_model=memory_model,
            **kwargs
        )

        self.validation_errors = 0
```

```
    async def _monitor_recv(self):
        while True:
            await FallingEdge(self.clock)

            if self._detect_handshake():
                packet = GAXIPacket(self.field_config)
                packet.start_time = get_sim_time('ns')

                # Use unified data collection
                data_dict = self._get_data_dict()
                self._finish_packet(get_sim_time('ns'), packet,
data_dict)

                # Validate against memory
                await self._validate_transaction(packet)

    async def _validate_transaction(self, packet):
        """Validate transaction against expected memory contents"""
        if hasattr(packet, 'cmd') and packet.cmd == 1:  # Read
            # Validate read data against memory
            success, expected_data = self.handle_memory_read(packet)
            if success and hasattr(packet, 'data'):
                if packet.data != expected_data:
                    self.validation_errors += 1
                    self.log.error(f"Data mismatch: expected
0x{expected_data:X}, "
                                   f"got 0x{packet.data:X}")

        elif hasattr(packet, 'cmd') and packet.cmd == 2:  # Write
            # Store write data in memory
            self.handle_memory_write(packet)

    def _detect_handshake(self):
        """Detect valid/ready handshake"""
        return (hasattr(self, 'valid_sig') and hasattr(self,
'ready_sig') and
                self.valid_sig.value == 1 and self.ready_sig.value ==
1)

    def get_validation_stats(self):
        """Get validation-specific statistics"""
        base_stats = self.get_base_stats()
        base_stats['validation_errors'] = self.validation_errors
        return base_stats
```

6.5.3 Protocol Violation Monitor

```
class ProtocolViolationMonitor(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
**kwargs):
        super().__init__(
            dut=dut,
            title=title,
            prefix=prefix,
            clock=clock,
            field_config=field_config,
            protocol_type='gaxi_master',
            **kwargs
        )

        self.protocol_violations = []
        self.x_z_violations = 0

    async def _monitor_recv(self):
        while True:
            await RisingEdge(self.clock)

            # Check for protocol violations
            violations = self._check_protocol_violations()
            if violations:
                self.protocol_violations.extend(violations)
                for violation in violations:
                    self.log.warning(f"Protocol violation:
{violation}")

            # Check for X/Z values
            if self._check_x_z_values():
                self.x_z_violations += 1
                self.log.error("X/Z values detected in signals")

            # Normal transaction monitoring
            if self._detect_valid_transaction():
                packet = GAXIPacket(self.field_config)
                packet.start_time = get_sim_time('ns')

                data_dict = self._get_data_dict()
                self._finish_packet(get_sim_time('ns'), packet,
data_dict)

    def _check_protocol_violations(self):
        """Check for protocol-specific violations"""
        violations = []

        # Check valid/ready protocol
```

```
        if (hasattr(self, 'valid_sig') and hasattr(self,
'ready_sig')):
            valid_val = self.valid_sig.value
            ready_val = self.ready_sig.value

            # Add custom protocol checks here
            # Example: Check for invalid signal combinations

        return violations

    def _check_x_z_values(self):
        """Check for X/Z values in critical signals"""
        try:
            critical_signals = [self.valid_sig, self.ready_sig]
            if hasattr(self, 'data_sig'):
                critical_signals.append(self.data_sig)

            for signal in critical_signals:
                if signal and not signal.value.is_resolvable:
                    return True
            return False
        except:
            return True

    def _detect_valid_transaction(self):
        """Detect valid transaction for normal monitoring"""
        return (hasattr(self, 'valid_sig') and hasattr(self,
'ready_sig') and
                self.valid_sig.value == 1 and self.ready_sig.value ==
1)

    def get_violation_stats(self):
        """Get protocol violation statistics"""
        base_stats = self.get_base_stats()
        base_stats.update({
            'protocol_violations': len(self.protocol_violations),
            'x_z_violations': self.x_z_violations,
            'violation_details': self.protocol_violations.copy()
        })
        return base_stats
```

6.5.4 Statistics Collection Monitor

```
class StatisticsMonitor(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
**kwargs):
        super().__init__(
            dut=dut,
            title=title,
```

```
            prefix=prefix,
            clock=clock,
            field_config=field_config,
            protocol_type='gaxi_master',
            **kwargs
        )

        self.transaction_latencies = []
        self.throughput_samples = []
        self.last_transaction_time = 0

    async def _monitor_recv(self):
        while True:
            await FallingEdge(self.clock)
            current_time = get_sim_time('ns')

            if self._detect_handshake():
                # Calculate inter-transaction time
                if self.last_transaction_time > 0:
                    inter_time = current_time -
self.last_transaction_time
                    self.throughput_samples.append(1e9 / inter_time)
# TPS

                self.last_transaction_time = current_time

                # Create and process packet
                packet = GAXIPacket(self.field_config)
                packet.start_time = current_time

                data_dict = self._get_data_dict()
                self._finish_packet(current_time, packet, data_dict)

                # Calculate latency if start time available
                if hasattr(packet, 'start_time') and hasattr(packet,
'end_time'):
                    latency = packet.end_time - packet.start_time
                    self.transaction_latencies.append(latency)

    def _detect_handshake(self):
        return (hasattr(self, 'valid_sig') and hasattr(self,
'ready_sig') and
                self.valid_sig.value == 1 and self.ready_sig.value ==
1)

    def get_performance_stats(self):
        """Get performance statistics"""
        base_stats = self.get_base_stats()
```

```
        if self.transaction_latencies:
            base_stats['average_latency_ns'] =
sum(self.transaction_latencies) / len(self.transaction_latencies)
            base_stats['min_latency_ns'] =
min(self.transaction_latencies)
            base_stats['max_latency_ns'] =
max(self.transaction_latencies)

        if self.throughput_samples:
            base_stats['average_throughput_tps'] =
sum(self.throughput_samples) / len(self.throughput_samples)
            base_stats['peak_throughput_tps'] =
max(self.throughput_samples)

        return base_stats
```

6.6 Integration with CocoTB

6.6.1 Callback System

```
# GAXIMonitorBase uses standard cocotb callback system
monitor.add_callback(transaction_handler)

def transaction_handler(packet):
    """Called automatically when packet is received"""
    print(f"Transaction: {packet.formatted()}")
```

6.6.2 Queue Management

```
# Standard cocotb queue access
packets = monitor._recvQ  # Direct access (not recommended)
packets = monitor.get_observed_packets()  # Preferred method
```

6.6.3 Signal Handling

```
# Automatic signal resolution from GAXIComponentBase
# Supports both automatic discovery and manual mapping
monitor = CustomMonitor(dut, "Monitor", "", clock, field_config,
                        signal_map={'valid': 'custom_valid'})
```

6.7 Error Handling

6.7.1 Missing Signals

```
# GAXIMonitorBase handles missing signals gracefully
try:
    data_dict = self._get_data_dict()
except AttributeError as e:
```

```
    self.log.warning(f"Signal access error: {e}")
    # Continue with partial data
```

6.7.2 Memory Operations

```
# Memory operations return success status
success = self.handle_memory_write(packet)
if not success:
    self.log.warning("Memory write failed - continuing monitoring")
```

6.7.3 Data Collection Errors

```
# Data collection handles X/Z values and signal errors
data_dict = self._get_data_dict()
# X/Z values returned as -1, handled automatically
```

6.8 Best Practices

6.8.1 1. Use Unified Methods

```
# Prefer unified methods over custom implementations
data_dict = self._get_data_dict()           # Not custom collection
self._finish_packet(time, packet, data)     # Not custom finishing
```

6.8.2 2. Handle Signal Errors Gracefully

```
def safe_signal_access(self):
    try:
        return self._get_data_dict()
    except AttributeError:
        return {}  # Return empty dict on signal errors
```

6.8.3 3. Use Memory Integration When Available

```
if self.memory_model:
    success = self.handle_memory_write(packet)
    if not success:
        self.log.warning("Memory operation failed")
```

6.8.4 4. Monitor Statistics Regularly

```
# Check statistics periodically
if packet_count % 100 == 0:
    stats = self.get_base_stats()
    print(f"Monitored {stats['observed_packets']} packets")
```

6.8.5 5. Implement Protocol-Specific Checks

```
class ProtocolSpecificMonitor(GAXIMonitorBase):
    def _validate_transaction(self, packet):
        """Add protocol-specific validation"""
```

```
        if hasattr(packet, 'addr') and packet.addr % 4 != 0:
            self.log.error("Unaligned address detected")
```

GAXIMonitorBase provides a solid foundation for GAXI monitoring components, eliminating code duplication while maintaining flexibility for protocol-specific monitoring requirements.

7 gaxi_packet.py

GAXI Packet class with minimal protocol-specific extensions to the base Packet class. Provides GAXI-specific randomizer handling while inheriting all field management, masking, pack/unpack, and formatting functionality.

7.1 Overview

The `GAXIPacket` class provides: - **Minimal extension** of base Packet class for GAXI protocol - **GAXI-specific randomizer handling** for master and slave interfaces - **Inherited functionality** from base Packet for all field operations - **Timing delay management** for valid and ready signals - **Protocol-specific delay calculations** using FlexRandomizer

All field management, masking, pack/unpack, and formatting functionality is inherited from the base Packet class without modification.

7.2 Class

7.2.1 GAXIPacket

```
class GAXIPacket(Packet):
    def __init__(self, field_config=None, master_randomizer=None,
                 slave_randomizer=None, **kwargs)
```

Parameters: - `field_config`: Field configuration (FieldConfig object or dict) - `master_randomizer`: Optional randomizer for master interface timing - `slave_randomizer`: Optional randomizer for slave interface timing
- `**kwargs`: Initial field values passed to parent Packet class

Inherited from Packet: - All field access via attributes (e.g., `packet.addr`, `packet.data`) - Automatic field validation and masking - FIFO packing/unpacking support - Rich formatting and comparison capabilities - Thread-safe field caching

7.3 GAXI-Specific Methods

7.3.1 Randomizer Management

7.3.1.1 set_master_randomizer(randomizer)

Set the master randomizer for valid delay generation.

Parameters: - `randomizer`: FlexRandomizer instance for master timing

```
from CocoTBFramework.components.shared.flex_randomizer import
FlexRandomizer

# Create master randomizer
master_randomizer = FlexRandomizer({
    'valid_delay': ([(0, 0), (1, 5), (10, 20)], [0.6, 0.3, 0.1])
})

packet.set_master_randomizer(master_randomizer)
```

7.3.1.2 set_slave_randomizer(randomizer)

Set the slave randomizer for ready delay generation.

Parameters: - `randomizer`: FlexRandomizer instance for slave timing

```
# Create slave randomizer
slave_randomizer = FlexRandomizer({
    'ready_delay': ([(0, 1), (2, 8), (9, 30)], [0.5, 0.3, 0.2])
})

packet.set_slave_randomizer(slave_randomizer)
```

7.3.2 Delay Generation

7.3.2.1 get_master_delay()

Get the delay for the master interface (valid delay).

Returns: Delay in cycles (0 if no randomizer)

```
# Get valid delay for master
valid_delay = packet.get_master_delay()
print(f"Master should wait {valid_delay} cycles before asserting
valid")
```

7.3.2.2 get_slave_delay()

Get the delay for the slave interface (ready delay).

Returns: Delay in cycles (0 if no randomizer)

```
# Get ready delay for slave
ready_delay = packet.get_slave_delay()
print(f"Slave should wait {ready_delay} cycles before asserting
ready")
```

7.4 Usage Patterns

7.4.1 Basic Packet Creation

```
from CocoTBFramework.components.gaxi.gaxi_packet import GAXIPacket
from CocoTBFramework.components.shared.field_config import FieldConfig

# Create field configuration
field_config = FieldConfig()
field_config.add_field(FieldDefinition("addr", 32, format="hex"))
field_config.add_field(FieldDefinition("data", 32, format="hex"))
field_config.add_field(FieldDefinition("cmd", 4, format="hex"))

# Create packet with initial values
packet = GAXIPacket(
    field_config=field_config,
    addr=0x1000,
    data=0xDEADBEEF,
    cmd=0x2  # WRITE command
)

# Access fields directly (inherited from base Packet)
print(f"Address: 0x{packet.addr:X}")
print(f"Data: 0x{packet.data:X}")
print(f"Command: {packet.cmd}")
```

7.4.2 Packet with Timing Randomizers

```
from CocoTBFramework.components.shared.flex_randomizer import
FlexRandomizer

# Create randomizers for timing
master_randomizer = FlexRandomizer({
    'valid_delay': ([(0, 0), (1, 3), (5, 10)], [0.7, 0.2, 0.1])
})

slave_randomizer = FlexRandomizer({
    'ready_delay': ([(0, 0), (1, 5)], [0.8, 0.2])
})

# Create packet with randomizers
packet = GAXIPacket(
    field_config=field_config,
    master_randomizer=master_randomizer,
    slave_randomizer=slave_randomizer,
    addr=0x2000,
    data=0xCAFEBABE
```

```
)

# Get timing delays
valid_delay = packet.get_master_delay()
ready_delay = packet.get_slave_delay()

print(f"Master valid delay: {valid_delay} cycles")
print(f"Slave ready delay: {ready_delay} cycles")
```

7.4.3 Field Operations (Inherited)

```
# All field operations inherited from base Packet class

# Field validation and masking (automatic)
packet.addr = 0x123456789  # Automatically masked to 32 bits ->
0x23456789
packet.data = 0xDEADBEEF   # Valid 32-bit value

# FIFO packing/unpacking (inherited)
fifo_data = packet.pack_for_fifo()
print(f"FIFO data: {fifo_data}")

# Create new packet from FIFO data
new_packet = GAXIPacket(field_config)
new_packet.unpack_from_fifo(fifo_data)

# Formatting (inherited)
print(packet.formatted())          # Detailed format
print(packet.formatted(compact=True))  # Compact format
```

7.4.4 Transaction Timing Integration

```
import cocotb
from cocotb.triggers import RisingEdge, Timer

class TimedGAXIMaster:
    def __init__(self, dut, clock, field_config):
        self.dut = dut
        self.clock = clock
        self.field_config = field_config

        # Create timing randomizers
        self.master_randomizer = FlexRandomizer({
            'valid_delay': ([(0, 0), (1, 5)], [0.8, 0.2])
        })

    async def send_packet(self, **field_values):
        """Send packet with randomized timing"""
```

```
        # Create packet with randomizer
        packet = GAXIPacket(
            field_config=self.field_config,
            master_randomizer=self.master_randomizer,
            **field_values
        )

        # Get timing delay
        valid_delay = packet.get_master_delay()

        # Apply delay before driving
        if valid_delay > 0:
            await Timer(valid_delay, units='clk')

        # Drive packet fields
        self.dut.addr.value = packet.addr
        self.dut.data.value = packet.data
        self.dut.valid.value = 1

        # Wait for handshake
        while self.dut.ready.value != 1:
            await RisingEdge(self.clock)

        await RisingEdge(self.clock)
        self.dut.valid.value = 0

        return packet

# Usage
master = TimedGAXIMaster(dut, clock, field_config)
packet = await master.send_packet(addr=0x1000, data=0x12345678,
cmd=0x2)
```

7.4.5 Slave Timing Integration

```
class TimedGAXISlave:
    def __init__(self, dut, clock, field_config):
        self.dut = dut
        self.clock = clock
        self.field_config = field_config

        # Create slave timing randomizer
        self.slave_randomizer = FlexRandomizer({
            'ready_delay': ([(0, 0), (1, 3), (5, 10)], [0.6, 0.3,
0.1])
        })

    async def receive_packet(self):
        """Receive packet with randomized ready timing"""
```

```
        # Wait for valid
        while self.dut.valid.value != 1:
            await RisingEdge(self.clock)

        # Create packet for timing
        packet = GAXIPacket(
            field_config=self.field_config,
            slave_randomizer=self.slave_randomizer
        )

        # Get ready delay
        ready_delay = packet.get_slave_delay()

        # Apply delay before asserting ready
        if ready_delay > 0:
            await Timer(ready_delay, units='clk')

        # Assert ready and capture data
        self.dut.ready.value = 1
        await RisingEdge(self.clock)

        # Capture packet data
        packet.addr = int(self.dut.addr.value)
        packet.data = int(self.dut.data.value)
        if hasattr(self.dut, 'cmd'):
            packet.cmd = int(self.dut.cmd.value)

        # Deassert ready
        self.dut.ready.value = 0

        return packet

# Usage
slave = TimedGAXISlave(dut, clock, field_config)
received_packet = await slave.receive_packet()
```

7.4.6 Batch Packet Creation

```
def create_test_packets(field_config, count=10):
    """Create a batch of test packets with varied data"""
    packets = []

    # Create common randomizers
    master_randomizer = FlexRandomizer({
        'valid_delay': ([(0, 0), (1, 2)], [0.9, 0.1])
    })

    slave_randomizer = FlexRandomizer({
        'ready_delay': ([(0, 1), (2, 5)], [0.7, 0.3])
```

```
    })

    for i in range(count):
        packet = GAXIPacket(
            field_config=field_config,
            master_randomizer=master_randomizer,
            slave_randomizer=slave_randomizer,
            addr=0x1000 + i*4,
            data=0x10000000 + i,
            cmd=0x2 if i % 2 == 0 else 0x1  # Alternate READ/write
        )
        packets.append(packet)

    return packets

# Usage
test_packets = create_test_packets(field_config, count=20)
for i, packet in enumerate(test_packets):
    print(f"Packet {i}: {packet.formatted(compact=True)}")
    print(f"  Master delay: {packet.get_master_delay()}")
    print(f"  Slave delay: {packet.get_slave_delay()}")
```

7.4.7 Packet Comparison and Validation

```
def validate_packet_sequence(sent_packets, received_packets):
    """Validate that received packets match sent packets"""
    assert len(sent_packets) == len(received_packets), \
        f"Packet count mismatch: sent {len(sent_packets)}, received
{len(received_packets)}"

    for i, (sent, received) in enumerate(zip(sent_packets,
received_packets)):
        # Use inherited comparison (ignores timing fields)
        if sent != received:
            print(f"Packet {i} mismatch:")
            print(f"  Sent:     {sent.formatted(compact=True)}")
            print(f"  Received: {received.formatted(compact=True)}")
            assert False, f"Packet {i} data mismatch"
        else:
            print(f"Packet {i}: ✓ Match")

    print(f"All {len(sent_packets)} packets validated successfully")

# Usage
sent_packets = create_test_packets(field_config, 10)
# ... send packets through DUT ...
received_packets = capture_received_packets()
validate_packet_sequence(sent_packets, received_packets)
```

7.4.8 Multi-Field Packets

```
# Create complex field configuration
complex_field_config = FieldConfig()
complex_field_config.add_field(FieldDefinition("addr", 32,
format="hex"))
complex_field_config.add_field(FieldDefinition("data", 64,
format="hex"))
complex_field_config.add_field(FieldDefinition("cmd", 4,
format="hex"))
complex_field_config.add_field(FieldDefinition("id", 8, format="hex"))
complex_field_config.add_field(FieldDefinition("size", 3,
format="hex"))

# Create packet with all fields
complex_packet = GAXIPacket(
    field_config=complex_field_config,
    addr=0x12345678,
    data=0xDEADBEEFCAFEBABE,
    cmd=0x3,
    id=0x55,
    size=0x4
)

# All field operations work automatically
print(f"Total packet bits: {complex_packet.get_total_bits()}")
print(f"Formatted packet:\n{complex_packet.formatted()}")

# FIFO operations handle all fields
fifo_data = complex_packet.pack_for_fifo()
print(f"FIFO data: {fifo_data}")
```

7.4.9 Randomizer Delay Caching

```
def test_delay_caching():
    """Test that delays are cached until randomizer is reset"""
    packet = GAXIPacket(field_config)

    # Set randomizer
    randomizer = FlexRandomizer({
        'valid_delay': ([(1, 5)], [1.0])
    })
    packet.set_master_randomizer(randomizer)

    # First call generates and caches delay
    delay1 = packet.get_master_delay()

    # Subsequent calls return cached value
    delay2 = packet.get_master_delay()
```

```
    delay3 = packet.get_master_delay()

    assert delay1 == delay2 == delay3, "Delay should be cached"
    print(f"Cached delay: {delay1} cycles")

    # Setting new randomizer resets cache
    new_randomizer = FlexRandomizer({
        'valid_delay': ([(10, 20)], [1.0])
    })
    packet.set_master_randomizer(new_randomizer)

    # New delay generated
    new_delay = packet.get_master_delay()
    print(f"New delay after randomizer change: {new_delay} cycles")

    # This new delay is now cached
    cached_new_delay = packet.get_master_delay()
    assert new_delay == cached_new_delay, "New delay should be cached"

test_delay_caching()
```

7.5 Error Handling

7.5.1 Field Validation Errors

```
# Field validation handled by base Packet class
try:
    packet = GAXIPacket(field_config, addr="invalid")
except ValueError as e:
    print(f"Invalid field value: {e}")
```

7.5.2 Missing Randomizers

```
# Graceful handling when randomizers not set
packet = GAXIPacket(field_config)

# Returns 0 when no randomizer
master_delay = packet.get_master_delay()  # Returns 0
slave_delay = packet.get_slave_delay()    # Returns 0

print(f"Delays without randomizers: master={master_delay},
slave={slave_delay}")
```

7.5.3 Randomizer Errors

```
# Handle randomizer generation errors
try:
    delay = packet.get_master_delay()
except Exception as e:
```

```
    print(f"Randomizer error: {e}")
    delay = 0  # Use default delay
```

7.6 Best Practices

7.6.1 1. Use Appropriate Randomizers for Each Interface

```
# Master randomizer for valid delays
master_randomizer = FlexRandomizer({'valid_delay': ([(0, 5)], [1.0])})

# Slave randomizer for ready delays
slave_randomizer = FlexRandomizer({'ready_delay': ([(0, 3)], [1.0])})

packet = GAXIPacket(field_config,
                    master_randomizer=master_randomizer,
                    slave_randomizer=slave_randomizer)
```

7.6.2 2. Leverage Inherited Functionality

```
# Use inherited field operations
packet.addr = 0x1000           # Field validation automatic
fifo_data = packet.pack_for_fifo()  # FIFO conversion automatic
formatted = packet.formatted()      # Rich formatting automatic
```

7.6.3 3. Cache Timing Delays Appropriately

```
# Delays are cached per packet - appropriate for single transaction
master_delay = packet.get_master_delay()  # Generated and cached
same_delay = packet.get_master_delay()    # Returns cached value

# Create new packet for new transaction with fresh delays
new_packet = GAXIPacket(field_config, master_randomizer=randomizer)
new_delay = new_packet.get_master_delay()  # Fresh delay
```

7.6.4 4. Use Timing Integration in Components

```
# Integrate timing into master/slave components
class TimingAwareMaster:
    def create_packet(self, **fields):
        return GAXIPacket(self.field_config,
                          master_randomizer=self.randomizer,
                          **fields)

    async def send(self, packet):
        delay = packet.get_master_delay()
        # Apply delay and send
```

7.6.5 5. Validate Packet Integrity

```
# Use inherited comparison for validation
assert sent_packet == received_packet, "Packet data mismatch"

# Use inherited formatting for debugging
if sent_packet != received_packet:
    print(f"Sent: {sent_packet.formatted(compact=True)}")
    print(f"Received: {received_packet.formatted(compact=True)}")
```

The GAXIPacket provides a minimal, focused extension to the base Packet class specifically for GAXI protocol timing requirements while leveraging all the robust field management, validation, and formatting capabilities of the base packet infrastructure.

8 gaxi_sequence.py

Enhanced GAXI sequence generator leveraging shared infrastructure for creating complex test patterns with dependency tracking, randomization, and built-in packet field masking.

8.1 Overview

The `GAXISequence` class provides: - **Enhanced sequence generation** using existing infrastructure - **FlexRandomizer integration** for value generation and constraints - **Built-in packet field masking** (no custom implementation needed) - **Standard dependency tracking** patterns - **Performance optimization** for large field configurations - **Multiple sequence types** (burst, pattern, randomized, dependency chains)

This simplified version eliminates custom field masking and uses existing infrastructure more effectively while preserving all existing APIs.

8.2 Class

8.2.1 GAXISequence

```
class GAXISequence:
    def __init__(self, name="basic", field_config=None,
packet_class=None, log=None)
```

Parameters: - `name`: Sequence name for identification - `field_config`: Field configuration (FieldConfig object or dictionary) - `packet_class`: Packet class to use (defaults to GAXIPacket) - `log`: Logger instance

Performance Note: The class automatically detects large field configurations (>50 bits) and uses optimized initialization to avoid performance issues.

8.3 Core Methods

8.3.1 Randomization Setup

8.3.1.1 set_randomizer(constraints_dict)

Set up randomizer for field value generation using FlexRandomizer.

Parameters: - `constraints_dict`: Dictionary of field constraints for FlexRandomizer

Returns: Self for method chaining

```
sequence = GAXISequence("randomized_test", field_config)

# Set up randomizer with constraints
sequence.set_randomizer({
    'data': ([(0x0000, 0xFFFF), (0x10000, 0x1FFFF)], [0.7, 0.3]),
    'addr': ([(0, 1023)], [1.0])
})
```

8.3.2 Transaction Management

8.3.2.1 add_transaction(field_values=None, delay=0, depends_on=None)

Add a transaction to the sequence with automatic field masking.

Parameters: - `field_values`: Dictionary of field values (automatically masked by Packet) - `delay`: Delay after this transaction - `depends_on`: Index of transaction this depends on (None for no dependency)

Returns: Index of added transaction for dependency tracking

```
# Add basic transaction
index = sequence.add_transaction({
    'addr': 0x1000,
    'data': 0xDEADBEEF,
    'cmd': 0x2
}, delay=5)

# Add transaction with dependency
dependent_index = sequence.add_transaction({
    'addr': 0x2000,
    'data': 0xCAFEBABE
}, depends_on=index)
```

8.3.2.2 add_data_transaction(data, delay=0, depends_on=None)

Add a simple data transaction.

Parameters: - `data`: Data value (automatically masked by Packet class) - `delay`: Delay after transaction - `depends_on`: Index of transaction this depends on

Returns: Index of added transaction

```
# Add data-only transactions
seq_index = sequence.add_data_transaction(0x12345678, delay=2)
next_index = sequence.add_data_transaction(0x87654321,
depends_on=seq_index)
```

8.3.3 Pattern Generation

8.3.3.1 add_burst(count, start_data=0, data_step=1, delay=0, dependency_chain=False)

Add a burst of transactions with incrementing data.

Parameters: - `count`: Number of transactions in burst - `start_data`: Starting data value - `data_step`: Step between data values - `delay`: Delay between transactions - `dependency_chain`: If True, each transaction depends on the previous

Returns: List of transaction indexes

```
# Add burst without dependencies
burst_indexes = sequence.add_burst(
    count=8,
    start_data=0x1000,
    data_step=0x100,
    delay=1
)

# Add burst with dependency chain
chain_indexes = sequence.add_burst(
    count=5,
    start_data=0x2000,
    data_step=4,
    dependency_chain=True  # Each depends on previous
)
```

8.3.3.2 add_pattern(pattern_name, data_width=32, delay=0)

Add common test patterns.

Parameters: - `pattern_name`: Type of pattern (‘walking_ones’, ‘walking_zeros’, ‘alternating’) - `data_width`: Width of data field - `delay`: Delay between pattern transactions

Returns: List of transaction indexes

```
# Add walking ones pattern
ones_indexes = sequence.add_pattern('walking_ones', data_width=32,
```

```
delay=1)

# Add walking zeros pattern
zeros_indexes = sequence.add_pattern('walking_zeros', data_width=16,
delay=2)

# Add alternating bit patterns
alt_indexes = sequence.add_pattern('alternating', data_width=32,
delay=0)
```

8.3.4 Randomized Transactions

8.3.4.1 add_randomized_transaction(delay=0, depends_on=None, field_overrides=None)

Add a transaction with randomized field values using FlexRandomizer.

Parameters: - `delay`: Delay after transaction - `depends_on`: Index of transaction this depends on - `field_overrides`: Dictionary of field values to override random values

Returns: Index of added transaction

```
# Must set randomizer first
sequence.set_randomizer({
    'addr': ([(0x1000, 0x8000)], [1.0]),
    'data': ([(0, 0xFFFFFFFF)], [1.0])
})

# Add randomized transaction
rand_index = sequence.add_randomized_transaction(
    delay=3,
    field_overrides={'cmd': 0x2}  # Override command to WRITE
)
```

8.3.4.2 generate_packets_with_randomization(count)

Generate packets with randomized values using FlexRandomizer.

Parameters: - `count`: Number of packets to generate

Returns: List of packets with randomized field values

```
# Generate random test packets
random_packets =
sequence.generate_packets_with_randomization(count=20)
for packet in random_packets:
    print(f"Random: {packet.formatted(compact=True)}")
```

8.3.5 Backward Compatibility Methods

8.3.5.1 add_data_value(data, delay=0)

Add a transaction with a data value - backward compatibility.

```
index = sequence.add_data_value(0xDEADBEEF, delay=5)
```

8.3.5.2 add_data_value_with_dependency(data, delay=0, depends_on_index=None, dependency_type="after")

Add a data transaction that depends on completion of a previous transaction.

```
base_index = sequence.add_data_value(0x1000)
dep_index = sequence.add_data_value_with_dependency(0x2000,
depends_on_index=base_index)
```

8.3.5.3 add_data_incrementing(count, data_start=0, data_step=1, delay=0)

Add transactions with incrementing data values.

Returns: Tuple of (self, list of indexes) for method chaining

```
seq, indexes = sequence.add_data_incrementing(
    count=10,
    data_start=0x1000,
    data_step=0x100,
    delay=2
)
```

8.3.6 Pattern Convenience Methods

8.3.6.1 add_walking_ones(data_width=32, delay=0)

Add transactions with walking ones pattern.

```
ones_indexes = sequence.add_walking_ones(data_width=32, delay=1)
```

8.3.6.2 add_walking_zeros(data_width=32, delay=0)

Add transactions with walking zeros pattern.

```
zeros_indexes = sequence.add_walking_zeros(data_width=16, delay=2)
```

8.3.6.3 add_alternating_bits(data_width=32, delay=0)

Add transactions with alternating bit patterns.

```
alt_indexes = sequence.add_alternating_bits(data_width=24, delay=1)
```

8.3.7 Dependency Management

8.3.7.1 add_burst_with_dependencies(count, data_start=0, data_step=1, delay=0, dependency_spacing=1)

Add a burst where each transaction depends on a previous one.

Parameters: - `count`: Number of transactions - `data_start`: Starting data value - `data_step`: Step size between data values - `delay`: Delay between transactions - `dependency_spacing`: How many transactions back to depend on

Returns: Tuple of (self, list of indexes)

```
# Each transaction depends on the previous one
seq, chain_indexes = sequence.add_burst_with_dependencies(
    count=5,
    data_start=0x1000,
    dependency_spacing=1
)

# Each transaction depends on two transactions back
seq, spaced_indexes = sequence.add_burst_with_dependencies(
    count=10,
    dependency_spacing=2
)
```

8.3.7.2 add_dependency_chain(count, data_start=0, data_step=1, delay=0)

Add a chain where each transaction depends on the previous one.

```
seq, chain_indexes = sequence.add_dependency_chain(
    count=8,
    data_start=0x2000,
    data_step=4
)
```

8.4 Packet Generation

8.4.1 generate_packets(count=None)

Generate packets from the sequence with dependency information.

Parameters: - `count`: Number of packets to generate (None for all)

Returns: List of generated packets with dependency metadata

```
# Generate all packets
all_packets = sequence.generate_packets()
```

```
# Generate first 10 packets
first_packets = sequence.generate_packets(count=10)

# Each packet includes dependency metadata
for packet in all_packets:
    if hasattr(packet, 'depends_on_index'):
        print(f"Packet {packet.sequence_index} depends on
{packet.depends_on_index}")
```

8.5 Dependency Analysis

8.5.1 get_dependency_order()

Get the order in which transactions should be executed based on dependencies.

Returns: List of transaction indexes in dependency-resolved order

```
# Get execution order
execution_order = sequence.get_dependency_order()
print(f"Execute transactions in order: {execution_order}")

# Generate packets in dependency order
ordered_packets = []
all_packets = sequence.generate_packets()
for index in execution_order:
    ordered_packets.append(all_packets[index])
```

8.5.2 validate_dependencies()

Validate that all dependencies are resolvable.

Returns: True if valid, raises ValueError if invalid

```
try:
    sequence.validate_dependencies()
    print("All dependencies are valid")
except ValueError as e:
    print(f"Dependency validation failed: {e}")
```

8.5.3 get_dependency_graph()

Get a representation of the transaction dependencies.

Returns: Dictionary mapping transaction indexes to their dependencies

```
dep_graph = sequence.get_dependency_graph()
print(f"Dependencies: {dep_graph['dependencies']}")
print(f"Transaction count: {dep_graph['transaction_count']}")
```

8.6 Statistics and Information

8.6.1 get_stats()

Get sequence generation statistics.

Returns: Dictionary with comprehensive statistics

```
stats = sequence.get_stats()
print(f"Sequence length: {stats['sequence_length']}")
print(f"Dependencies: {stats['dependencies']}")
print(f"Dependency percentage: {stats.get('dependency_percentage',
0):.1f}%")
print(f"Uses randomization: {stats['uses_randomization']}")
```

8.6.2 reset()

Reset sequence to empty state.

```
sequence.reset()  # Clear all transactions and dependencies
```

8.6.3 __len__()

Return number of transactions in sequence.

```
transaction_count = len(sequence)
```

8.7 Usage Patterns

8.7.1 Basic Sequence Creation

```
from CocoTBFramework.components.gaxi.gaxi_sequence import GAXISequence
from CocoTBFramework.components.shared.field_config import FieldConfig

# Create field configuration
field_config = FieldConfig()
field_config.add_field(FieldDefinition("addr", 32, format="hex"))
field_config.add_field(FieldDefinition("data", 32, format="hex"))
field_config.add_field(FieldDefinition("cmd", 4, format="hex"))

# Create sequence
sequence = GAXISequence("basic_test", field_config)

# Add various transaction types
sequence.add_transaction({'addr': 0x1000, 'data': 0xDEADBEEF, 'cmd':
0x2})
sequence.add_data_transaction(0x12345678, delay=2)
sequence.add_burst(count=5, start_data=0x1000, data_step=0x100)
```

```python
# Generate packets
packets = sequence.generate_packets()
print(f"Generated {len(packets)} packets")
```

8.7.2 Randomized Sequence

```python
# Create sequence with randomization
sequence = GAXISequence("random_test", field_config)

# Set up randomizer
sequence.set_randomizer({
    'addr': ([(0x1000, 0x1FFF), (0x8000, 0x8FFF)], [0.7, 0.3]),
    'data': ([(0, 0xFFFF), (0x10000, 0xFFFFFFFF)], [0.5, 0.5]),
    'cmd': ([(0x1, 0x2)], [1.0])  # READ or WRITE only
})

# Add randomized transactions
for i in range(20):
    sequence.add_randomized_transaction(delay=random.randint(0, 5))

# Generate packets with random values
packets = sequence.generate_packets()
```

8.7.3 Pattern-Based Testing

```python
# Create pattern sequence
pattern_sequence = GAXISequence("pattern_test", field_config)

# Add different test patterns
pattern_sequence.add_walking_ones(data_width=32, delay=1)
pattern_sequence.add_walking_zeros(data_width=32, delay=1)
pattern_sequence.add_alternating_bits(data_width=32, delay=2)

# Add custom patterns
custom_patterns = [0xAAAAAAAA, 0x55555555, 0xCCCCCCCC, 0x33333333]
for pattern in custom_patterns:
    pattern_sequence.add_data_transaction(pattern, delay=1)

packets = pattern_sequence.generate_packets()
print(f"Pattern sequence has {len(packets)} packets")
```

8.7.4 Dependency Chain Testing

```python
# Create dependency sequence
dep_sequence = GAXISequence("dependency_test", field_config)

# Add initial transaction
```

```python
base_index = dep_sequence.add_transaction({
    'addr': 0x1000,
    'data': 0x1,
    'cmd': 0x2  # WRITE
})

# Add chain of dependent transactions
for i in range(1, 10):
    dep_index = dep_sequence.add_transaction({
        'addr': 0x1000 + i*4,
        'data': i,
        'cmd': 0x2
    }, depends_on=base_index + i - 1)  # Depends on previous

# Validate dependencies
dep_sequence.validate_dependencies()

# Get execution order
order = dep_sequence.get_dependency_order()
print(f"Execution order: {order}")

# Generate packets in dependency order
packets = dep_sequence.generate_packets()
```

8.7.5 Complex Mixed Sequence

```python
def create_complex_test_sequence(field_config):
    """Create a complex sequence with mixed transaction types"""
    sequence = GAXISequence("complex_test", field_config)

    # Set up randomizer for some transactions
    sequence.set_randomizer({
        'addr': ([(0x1000, 0x8000)], [1.0]),
        'data': ([(0, 0xFFFFFFFF)], [1.0])
    })

    # Phase 1: Initialization burst
    init_indexes = sequence.add_burst(
        count=4,
        start_data=0x10000000,
        data_step=0x100,
        delay=0
    )

    # Phase 2: Pattern testing (depends on init)
    pattern_start = sequence.add_walking_ones(data_width=32, delay=1)
    for idx in pattern_start[:3]:  # First 3 patterns depend on init
        sequence.sequence_data[idx] = (
```

```
                sequence.sequence_data[idx][0],  # field_values
                sequence.sequence_data[idx][1],  # delay
                init_indexes[-1]  # depends_on last init transaction
            )

        # Phase 3: Random transactions
        random_indexes = []
        for i in range(10):
            rand_idx = sequence.add_randomized_transaction(delay=2)
            random_indexes.append(rand_idx)

        # Phase 4: Cleanup (depends on random completion)
        cleanup_indexes = sequence.add_burst(
            count=3,
            start_data=0x99999999,
            dependency_chain=True
        )

        return sequence

# Create and use complex sequence
complex_seq = create_complex_test_sequence(field_config)
packets = complex_seq.generate_packets()

# Analyze sequence
stats = complex_seq.get_stats()
print(f"Complex sequence statistics: {stats}")
```

8.7.6 Factory Methods

The class provides factory methods for common sequence types:

8.7.6.1 create_burst_sequence(name, count, start_data=0, data_step=1, field_config=None, dependency_chain=False)

Create a burst sequence with incrementing data.

```
burst_seq = GAXISequence.create_burst_sequence(
    name="burst_test",
    count=16,
    start_data=0x1000,
    data_step=4,
    field_config=field_config,
    dependency_chain=True
)
```

8.7.6.2 create_pattern_sequence(name, pattern_name, data_width=32, field_config=None)

Create a sequence with test patterns.

```
pattern_seq = GAXISequence.create_pattern_sequence(
    name="walking_ones_test",
    pattern_name="walking_ones",
    data_width=32,
    field_config=field_config
)
```

8.7.6.3 create_randomized_sequence(name, constraints, count, field_config=None)

Create a sequence with randomized values.

```
random_seq = GAXISequence.create_randomized_sequence(
    name="random_test",
    constraints={
        'addr': ([(0x1000, 0x8000)], [1.0]),
        'data': ([(0, 0xFFFFFFFF)], [1.0])
    },
    count=50,
    field_config=field_config
)
```

8.7.6.4 create_dependency_chain(name="dependency_chain", count=5, data_start=0, data_step=1, delay=0)

Create a sequence with transactions forming a dependency chain.

```
chain_seq = GAXISequence.create_dependency_chain(
    name="chain_test",
    count=8,
    data_start=0x2000,
    data_step=8,
    delay=1
)
```

8.8 Performance Considerations

8.8.1 Large Field Configurations

For field configurations with >50 total bits, the class automatically uses optimized initialization:

```
# Automatically optimized for large configs
large_config = create_large_field_config()  # >50 bits
sequence = GAXISequence("large_test", large_config)  # Uses optimized
mode
```

8.8.2 Memory Usage

The class efficiently manages memory for large sequences:

```
# Efficient for large sequences
large_sequence = GAXISequence("large", field_config)
for i in range(10000):
    large_sequence.add_data_transaction(i)
# Memory usage remains reasonable
```

8.9 Error Handling

8.9.1 Dependency Validation

```
try:
    sequence.validate_dependencies()
except ValueError as e:
    print(f"Invalid dependencies: {e}")
    # Fix dependencies and retry
```

8.9.2 Randomizer Errors

```
try:
    sequence.add_randomized_transaction()
except ValueError as e:
    print(f"No randomizer set: {e}")
    # Set randomizer first
    sequence.set_randomizer(constraints)
```

8.9.3 Field Validation

```
# Field validation handled automatically by Packet class
sequence.add_transaction({'addr': 0x123456789})  # Automatically
masked
```

8.10 Best Practices

8.10.1 1. Use Factory Methods for Common Patterns

```
# Prefer factory methods
burst_seq = GAXISequence.create_burst_sequence("test", 10, 0x1000)
# Over manual creation
```

8.10.2 2. Validate Dependencies Early

```
# Validate after adding dependent transactions
sequence.add_burst_with_dependencies(count=5, dependency_spacing=1)
sequence.validate_dependencies()  # Catch errors early
```

8.10.33. Use Randomization for Stress Testing

```
# Set up realistic randomization constraints
sequence.set_randomizer({
    'addr': ([(0x1000, 0x7FFF)], [1.0]),  # Valid address range
    'data': ([(0, 0xFFFFFFFF)], [1.0])     # Full data range
})
```

8.10.44. Generate Packets in Dependency Order

```
# For dependency chains, use dependency order
order = sequence.get_dependency_order()
packets = sequence.generate_packets()
ordered_packets = [packets[i] for i in order]
```

8.10.55. Monitor Sequence Statistics

```
# Check sequence health
stats = sequence.get_stats()
if stats['dependency_percentage'] > 50:
    print("Warning: High dependency ratio may affect parallelism")
```

The GAXISequence provides a powerful and flexible framework for creating comprehensive test patterns with dependency management, randomization, and performance optimization for GAXI protocol verification.

9 gaxi_slave.py

GAXI Slave with integrated structured pipeline that maintains all existing functionality and timing while adding better debugging and error recovery through structured pipeline phases.

9.1 Overview

The `GAXISlave` class provides: - **Structured receive pipeline** with enhanced debugging and error recovery - **Active ready signal driving** with timing control - **Mode-specific data capture** (skid, fifo_mux, fifo_flop) - **Memory model integration** using base MemoryModel directly - **Comprehensive statistics** including pipeline performance - **Optional pipeline debugging** for troubleshooting

Inherits from GAXIMonitorBase which provides common monitoring functionality and signal resolution.

9.2 Class

9.2.1 GAXISlave

```
class GAXISlave(GAXIMonitorBase):
    def __init__(self, dut, title, prefix, clock, field_config,
```

```
                  timeout_cycles=1000, mode='skid', bus_name='',
pkt_prefix='',
                  multi_sig=False, randomizer=None, memory_model=None,
                  log=None, super_debug=False, pipeline_debug=False,
                  signal_map=None, protocol_type='gaxi_slave',
**kwargs)
```

Parameters: - `dut`: Device under test - `title`: Component title/name - `prefix`: Bus prefix - `clock`: Clock signal - `field_config`: Field configuration - `timeout_cycles`: Timeout for operations (default: 1000) - `mode`: GAXI mode (‘skid’, ‘fifo_mux’, ‘fifo_flop’) - `bus_name`: Bus/channel name - `pkt_prefix`: Packet field prefix - `multi_sig`: Whether using multi-signal mode - `randomizer`: Optional randomizer for ready delays - `memory_model`: Optional memory model for transactions - `log`: Logger instance (required — raises ValueError if None; pass your TBBase logger) - `super_debug`: Enable detailed debugging - `pipeline_debug`: Enable pipeline phase debugging - `signal_map`: Optional manual signal mapping override - `**kwargs`: Additional arguments

9.3 Data Capture Modes

The GAXISlave supports different data capture timing based on the mode:

9.3.1 Skid Mode (`mode='skid'`)

- **Data capture**: Immediate (same cycle as handshake)
- **Use case**: Standard pipeline with no delay
- **Performance**: Fastest response time

9.3.2 FIFO MUX Mode (`mode='fifo_mux'`)

- **Data capture**: Immediate (same cycle as handshake)
- **Use case**: Multiplexed FIFO interface
- **Performance**: Fast response time

9.3.3 FIFO FLOP Mode (`mode='fifo_flop'`)

- **Data capture**: Delayed (one cycle after handshake)
- **Use case**: Registered FIFO interface
- **Performance**: One cycle latency for data capture

9.4 Core Methods

9.4.1 Bus Management

9.4.1.1 `async reset_bus()`

Reset bus with enhanced pipeline state management.

```
await slave.reset_bus()
```

9.4.2 Callback Management

9.4.2.1 add_callback(callback)

Add callback for received transactions.

Parameters: - `callback`: Function to call when transaction is received

```
def transaction_handler(packet):
    print(f"Received: {packet.formatted()}")

slave.add_callback(transaction_handler)
```

9.4.2.2 remove_callback(callback)

Remove callback.

```
slave.remove_callback(transaction_handler)
```

9.4.3 Data Access

9.4.3.1 get_observed_packets(count=None)

Get observed packets from the receive queue.

Parameters: - `count`: Number of packets to return (None = all)

Returns: List of observed packets

```
# Get all observed packets
all_packets = slave.get_observed_packets()

# Get last 5 packets
recent_packets = slave.get_observed_packets(5)
```

9.4.3.2 clear_observed_packets()

Clear the observed packets queue.

```
slave.clear_observed_packets()
```

9.4.4 Pipeline Control and Debugging

9.4.4.1 enable_pipeline_debug(enable=True)

Enable or disable pipeline debugging at runtime.

```
# Enable detailed pipeline logging
slave.enable_pipeline_debug(True)
```

```
# Disable for performance
slave.enable_pipeline_debug(False)
```

9.4.4.2 get_pipeline_stats()

Get pipeline-specific statistics.

Returns: Dictionary with pipeline statistics

```
pipeline_stats = slave.get_pipeline_stats()
print(f"Current state: {pipeline_stats['current_state']}")
print(f"Handshakes: {pipeline_stats['handshake_count']}")
print(f"Immediate captures: {pipeline_stats['immediate_captures']}")
print(f"Deferred captures: {pipeline_stats['deferred_captures']}")
```

9.4.4.3 get_pipeline_debug_info()

Get detailed pipeline debug information.

```
debug_info = slave.get_pipeline_debug_info()
print(f"Phase timings: {debug_info['phase_timings']}")
print(f"Mode: {debug_info['mode']}")
```

9.4.5 Statistics

9.4.5.1 get_stats()

Get comprehensive statistics including pipeline stats.

Returns: Dictionary containing all statistics

```
stats = slave.get_stats()
print(f"Monitor stats: {stats['slave_stats']}")
print(f"Pipeline stats: {stats['pipeline_stats']}")
print(f"Base stats: {stats}")
```

9.5 Pipeline Architecture

The GAXISlave uses a structured 3-phase receive pipeline:

9.5.1 Phase 1: Handle Pending Transactions

- Process deferred captures from fifo_flop mode
- Maintain exact original timing
- Enhanced debugging and statistics

9.5.2 Phase 2: Ready Timing Control

- Apply `ready_delay` from randomizer

- Deassert ready during delay periods
- Assert ready when ready to accept data
- Wait for falling edge for signal sampling

9.5.3 Phase 3: Transaction Processing

- Detect valid/ready handshake
- Create packet and capture timing
- Mode-specific data capture (immediate vs deferred)
- Memory operations and callback triggering

9.5.4 Pipeline State Tracking

```
# Pipeline states
"idle" → "monitor_start" → "cycle_start" → "phase1" →
"phase2" → "phase3" → "cycle_end" → "cycle_start" ...

# Error states
"error_recovery", "reset"
```

9.6 Usage Patterns

9.6.1 Basic Usage

```
import cocotb
from cocotb.triggers import RisingEdge
from CocoTBFramework.components.gaxi import GAXISlave
from CocoTBFramework.components.shared.field_config import FieldConfig

@cocotb.test()
async def test_gaxi_slave(dut):
    clock = dut.clk

    # Create field configuration
    field_config = FieldConfig()
    field_config.add_field(FieldDefinition("addr", 32, format="hex"))
    field_config.add_field(FieldDefinition("data", 32, format="hex"))

    # Create slave
    slave = GAXISlave(
        dut=dut,
        title="TestSlave",
        prefix="",
        clock=clock,
        field_config=field_config,
        mode='skid',
```

```
        log=log,              # Required
        pipeline_debug=True  # Enable pipeline debugging
    )

    # Add callback to process transactions
    def process_transaction(packet):
        print(f"Slave received: addr=0x{packet.addr:X},
data=0x{packet.data:X}")

    slave.add_callback(process_transaction)

    # Reset bus
    await slave.reset_bus()

    # Start monitoring (automatically started)
    # Transactions will be captured and processed via callbacks

    # Wait for some transactions
    await Timer(1000, units='ns')

    # Check received transactions
    packets = slave.get_observed_packets()
    print(f"Received {len(packets)} transactions")
```

9.6.2 Mode-Specific Configuration

```
# Skid mode - immediate capture
skid_slave = GAXISlave(
    dut=dut,
    title="SkidSlave",
    prefix="s_",
    clock=clock,
    field_config=field_config,
    mode='skid'  # Immediate data capture
)

# FIFO FLOP mode - delayed capture
flop_slave = GAXISlave(
    dut=dut,
    title="FlopSlave",
    prefix="f_",
    clock=clock,
    field_config=field_config,
    mode='fifo_flop'  # Delayed data capture
)
```

9.6.3 Advanced Configuration with Memory

```
from CocoTBFramework.components.shared.flex_randomizer import
FlexRandomizer
from CocoTBFramework.components.shared.memory_model import MemoryModel

# Create randomizer for ready delays
randomizer = FlexRandomizer({
    'ready_delay': ([(0, 1), (2, 8), (9, 30)], [0.6, 0.3, 0.1])
})

# Create memory model for transaction storage
memory = MemoryModel(num_lines=1024, bytes_per_line=4, log=log)

# Create slave with advanced configuration
slave = GAXISlave(
    dut=dut,
    title="AdvancedSlave",
    prefix="adv_",
    clock=clock,
    field_config=field_config,
    randomizer=randomizer,
    memory_model=memory,
    mode='fifo_mux',
    pipeline_debug=True,
    multi_sig=True
)
```

9.6.4 Memory-Integrated Processing

```
@cocotb.test()
async def test_memory_slave(dut):
    # Create memory model
    memory = MemoryModel(num_lines=256, bytes_per_line=4, log=log)

    # Create slave with memory
    slave = GAXISlave(
        dut=dut,
        title="MemorySlave",
        prefix="",
        clock=clock,
        field_config=field_config,
        memory_model=memory
    )

    # Memory operations happen automatically in the pipeline
    # Check memory contents after transactions
    await Timer(1000, units='ns')
```

```
    # Get memory statistics
    stats = slave.get_stats()
    if 'memory_stats' in stats:
        memory_stats = stats['memory_stats']
        print(f"Memory writes: {memory_stats['writes']}")
        print(f"Memory coverage:
{memory_stats['write_coverage']:.1%}")
```

9.6.5 Pipeline Performance Analysis

```
@cocotb.test()
async def test_pipeline_performance(dut):
    slave = GAXISlave(dut, "PerfSlave", "", clock, field_config,
                      pipeline_debug=True)

    # Run for a period
    await Timer(10000, units='ns')

    # Analyze pipeline performance
    pipeline_stats = slave.get_pipeline_stats()

    print(f"Pipeline Performance Analysis:")
    print(f"  Total handshakes: {pipeline_stats['handshake_count']}")
    print(f"  Immediate captures:
{pipeline_stats['immediate_captures']}")
    print(f"  Deferred captures:
{pipeline_stats['deferred_captures']}")
    print(f"  Memory operations:
{pipeline_stats['memory_operations']}")
    print(f"  Errors: {pipeline_stats['error_count']}")

    # Calculate efficiency metrics
    total_captures = (pipeline_stats['immediate_captures'] +
                      pipeline_stats['deferred_captures'])
    if total_captures > 0:
        immediate_rate = pipeline_stats['immediate_captures'] /
total_captures
        print(f"  Immediate capture rate: {immediate_rate:.1%}")

    # Get timing information
    debug_info = slave.get_pipeline_debug_info()
    for phase, timing in debug_info['phase_timings'].items():
        print(f"  {phase}: {timing}ns")
```

9.6.6 Ready Delay Testing

```
@cocotb.test()
async def test_ready_delays(dut):
    # Create randomizer with specific delay patterns
    randomizer = FlexRandomizer({
```

```
        'ready_delay': ([(0, 0), (1, 1), (5, 10)], [0.5, 0.3, 0.2])
    })

    slave = GAXISlave(
        dut=dut,
        title="DelayedSlave",
        prefix="",
        clock=clock,
        field_config=field_config,
        randomizer=randomizer,
        pipeline_debug=True
    )

    # Track ready signal behavior
    ready_assert_times = []
    ready_delays = []

    def track_ready_timing(packet):
        # This callback is called when transaction completes
        # Can analyze timing here
        pass

    slave.add_callback(track_ready_timing)

    # Monitor for a period
    await Timer(5000, units='ns')

    # Analyze ready delay effectiveness
    pipeline_stats = slave.get_pipeline_stats()
    print(f"Ready delay testing: {pipeline_stats['handshake_count']}
handshakes")
```

9.6.7 Callback-Based Processing

```
class TransactionProcessor:
    def __init__(self):
        self.received_count = 0
        self.data_sum = 0

    def process_transaction(self, packet):
        """Callback for processing received transactions"""
        self.received_count += 1
        self.data_sum += packet.data

        print(f"Transaction {self.received_count}: "
              f"addr=0x{packet.addr:X}, data=0x{packet.data:X}")

        # Perform application-specific processing
        if packet.addr >= 0x8000:
```

```
            print("  → High address region access")

        if packet.data == 0xDEADBEEF:
            print("  → Test pattern detected")

    def get_summary(self):
        avg_data = self.data_sum / self.received_count if
self.received_count > 0 else 0
        return {
            'received_count': self.received_count,
            'average_data': avg_data
        }

@cocotb.test()
async def test_callback_processing(dut):
    # Create processor
    processor = TransactionProcessor()

    # Create slave with callback
    slave = GAXISlave(dut, "CallbackSlave", "", clock, field_config)
    slave.add_callback(processor.process_transaction)

    # Run test
    await Timer(2000, units='ns')

    # Get processing summary
    summary = processor.get_summary()
    print(f"Processing summary: {summary}")
```

9.6.8 Error Handling and Recovery

```
@cocotb.test()
async def test_error_recovery(dut):
    slave = GAXISlave(dut, "ErrorSlave", "", clock, field_config,
                      pipeline_debug=True)

    # Callback that might cause errors
    def error_prone_callback(packet):
        if packet.data == 0xBADDATA:
            raise ValueError("Bad data detected")
        print(f"Processed: {packet.formatted()}")

    slave.add_callback(error_prone_callback)

    try:
        # Run test
        await Timer(1000, units='ns')

    except Exception as e:
```

```
        log.error(f"Test error: {e}")

        # Check pipeline state
        debug_info = slave.get_pipeline_debug_info()
        print(f"Pipeline state: {debug_info['current_state']}")

        # Pipeline should continue operating despite callback errors
        pipeline_stats = slave.get_pipeline_stats()
        print(f"Error count: {pipeline_stats['error_count']}")

        # Reset if needed
        await slave.reset_bus()
```

9.7 Error Handling

9.7.1 Signal Mapping Errors

```
try:
    slave = GAXISlave(dut, "Slave", "", clock, field_config)
except RuntimeError as e:
    # Try manual signal mapping
    signal_map = {'valid': 'custom_valid', 'ready': 'custom_ready'}
    slave = GAXISlave(dut, "Slave", "", clock, field_config,
                      signal_map=signal_map)
```

9.7.2 Memory Operation Errors

```
# Memory operations handled automatically with error logging
# Check memory statistics for error information
stats = slave.get_stats()
if 'memory_stats' in stats:
    memory_stats = stats['memory_stats']
    if memory_stats['boundary_violations'] > 0:
        log.warning(f"Memory boundary violations:
{memory_stats['boundary_violations']}")
```

9.7.3 Pipeline Errors

```
# Pipeline errors are tracked in statistics
pipeline_stats = slave.get_pipeline_stats()
if pipeline_stats['error_count'] > 0:
    log.warning(f"Pipeline errors detected:
{pipeline_stats['error_count']}")

    # Get detailed error information
    debug_info = slave.get_pipeline_debug_info()
    print(f"Current state: {debug_info['current_state']}")
```

9.8 Best Practices

9.8.1 1. Choose Appropriate Mode for DUT

```
# Match slave mode to DUT implementation
if dut_uses_registered_interface:
    mode = 'fifo_flop'  # Delayed capture
else:
    mode = 'skid'       # Immediate capture
```

9.8.2 2. Use Callbacks for Transaction Processing

```
# Prefer callbacks over polling
slave.add_callback(process_transaction)

# Avoid polling _recvQ directly
# packets = slave._recvQ  # Don't do this
packets = slave.get_observed_packets()  # Do this instead
```

9.8.3 3. Enable Pipeline Debugging During Development

```
slave = GAXISlave(..., pipeline_debug=True)   # Development
slave = GAXISlave(..., pipeline_debug=False)  # Production
```

9.8.4 4. Configure Ready Delays Appropriately

```
# For throughput testing
randomizer = FlexRandomizer({'ready_delay': ([(0, 0)], [1.0])})

# For realistic backpressure
randomizer = FlexRandomizer({
    'ready_delay': ([(0, 1), (2, 8)], [0.7, 0.3])
})
```

9.8.5 5. Monitor Pipeline Statistics

```
# Regular statistics monitoring
if transaction_count % 100 == 0:
    pipeline_stats = slave.get_pipeline_stats()
    if pipeline_stats['error_count'] > expected_threshold:
        handle_error_condition()
```

The GAXISlave provides comprehensive transaction reception capabilities with flexible timing control, mode-specific data capture, and extensive debugging support for thorough verification of GAXI protocol implementations.

10 GAXI Components Index

This directory contains the GAXI (Generic AXI) protocol components for the CocoTBFramework. GAXI provides a lightweight valid/ready handshake protocol for validating individual FIFO-based interfaces on very small internal blocks. Interfaces can carry data packed into fields within a single bus, or have many discrete signals — GAXI handles both.

10.1 Directory Structure

10.1.1 Core Components

- **gaxi_component_base.py** - Unified base class for all GAXI components
- **gaxi_master.py** - GAXI Master with integrated structured pipeline
- **gaxi_slave.py** - GAXI Slave with integrated structured pipeline
- **gaxi_monitor.py** - GAXI Monitor implementation
- **gaxi_monitor_base.py** - Base class for GAXI monitoring functionality

10.1.2 Data and Protocol Support

- **gaxi_packet.py** - GAXI Packet class with protocol-specific extensions
- **gaxi_sequence.py** - GAXI sequence generator for test patterns
- **gaxi_command_handler.py** - Enhanced command handler for transactions

10.1.3 Factory and Utilities

- **gaxi_factories.py** - Factory functions for creating GAXI components

10.2 Quick Start

10.2.1 Basic Usage

```
from CocoTBFramework.components.gaxi.gaxi_factories import
create_gaxi_system

# Create complete GAXI system (log is required by the underlying
components)
system = create_gaxi_system(dut, clock, log=log)
master = system['master']
slave = system['slave']

# Send data
await master.send(master.create_packet(data=0xDEADBEEF))
```

10.2.2 Advanced Usage

```
from CocoTBFramework.components.gaxi import GAXIMaster, GAXISlave,
GAXIMonitor
from CocoTBFramework.components.gaxi.gaxi_sequence import GAXISequence

# Create individual components (log is required)
master = GAXIMaster(dut, "TestMaster", "", clock, field_config,
log=log)
slave = GAXISlave(dut, "TestSlave", "", clock, field_config, log=log)
monitor = GAXIMonitor(dut, "Monitor", "", clock, field_config,
log=log)

# Create test sequence
sequence = GAXISequence("test_pattern", field_config)
sequence.add_burst(count=10, start_data=0x1000)
packets = sequence.generate_packets()
```

10.3 Component Overview

10.3.1 GAXIMaster

- Drives GAXI transactions with configurable timing
- Supports multi-signal and single-signal modes
- Integrated pipeline debugging and statistics
- Memory model integration for read/write operations

10.3.2 GAXISlave

- Receives GAXI transactions with configurable ready delays
- Supports different modes (skid, fifo_mux, fifo_flop)
- Automatic memory operations and response generation
- Pipeline debugging and error recovery

10.3.3 GAXIMonitor

- Observes GAXI transactions on master or slave side
- Protocol violation detection
- Statistics collection and reporting
- Integration with scoreboards

10.3.4 Supporting Classes

- **GAXIPacket**: Protocol-specific packet with field management
- **GAXISequence**: Test pattern generation with dependencies

- **GAXICommandHandler**: Transaction coordination and memory operations
- **GAXIComponentBase**: Common functionality for all components

10.4 Features

10.4.1 Signal Resolution

- Automatic signal discovery using pattern matching
- Manual signal mapping override capability
- Support for different signal naming conventions
- Multi-signal and single-signal field modes

10.4.2 Performance Optimization

- Cached signal references for high-performance operation
- Thread-safe operation for parallel testing
- Optimized data collection and driving strategies
- Reduced signal lookup overhead

10.4.3 Debugging Support

- Pipeline state tracking and transition logging
- Comprehensive statistics collection
- Protocol violation detection and reporting
- Memory operation tracking and validation

10.4.4 Flexibility

- Configurable field definitions and packet structures
- Multiple randomization modes and constraints
- Dependency tracking in test sequences
- Memory model integration for transaction processing

10.5 Integration

The GAXI components integrate seamlessly with: - **Shared Components**: Field configuration, memory models, statistics - **Scoreboards**: Automatic transaction recording and comparison - **Randomization**: FlexRandomizer for timing and data generation - **CocoTB**: Standard BusDriver/BusMonitor interfaces

10.6 Navigation

- **Overview** - Detailed component overview and architecture
- **Back to Components** - Return to components index
- **Back to CocoTBFramework** - Return to main framework index

11 GAXI Components Overview

The GAXI (Generic AXI) components provide a lightweight, valid/ready handshake protocol for validating individual FIFO-based interfaces on very small internal blocks. Interfaces can carry data packed into fields within a single bus, or they can have many discrete signals — GAXI handles both through its flexible field configuration system. These components are designed to eliminate code duplication while maintaining exact timing compatibility and providing enhanced debugging capabilities.

11.1 Architecture Overview

The GAXI component architecture follows a hierarchical design with shared base classes and unified infrastructure:

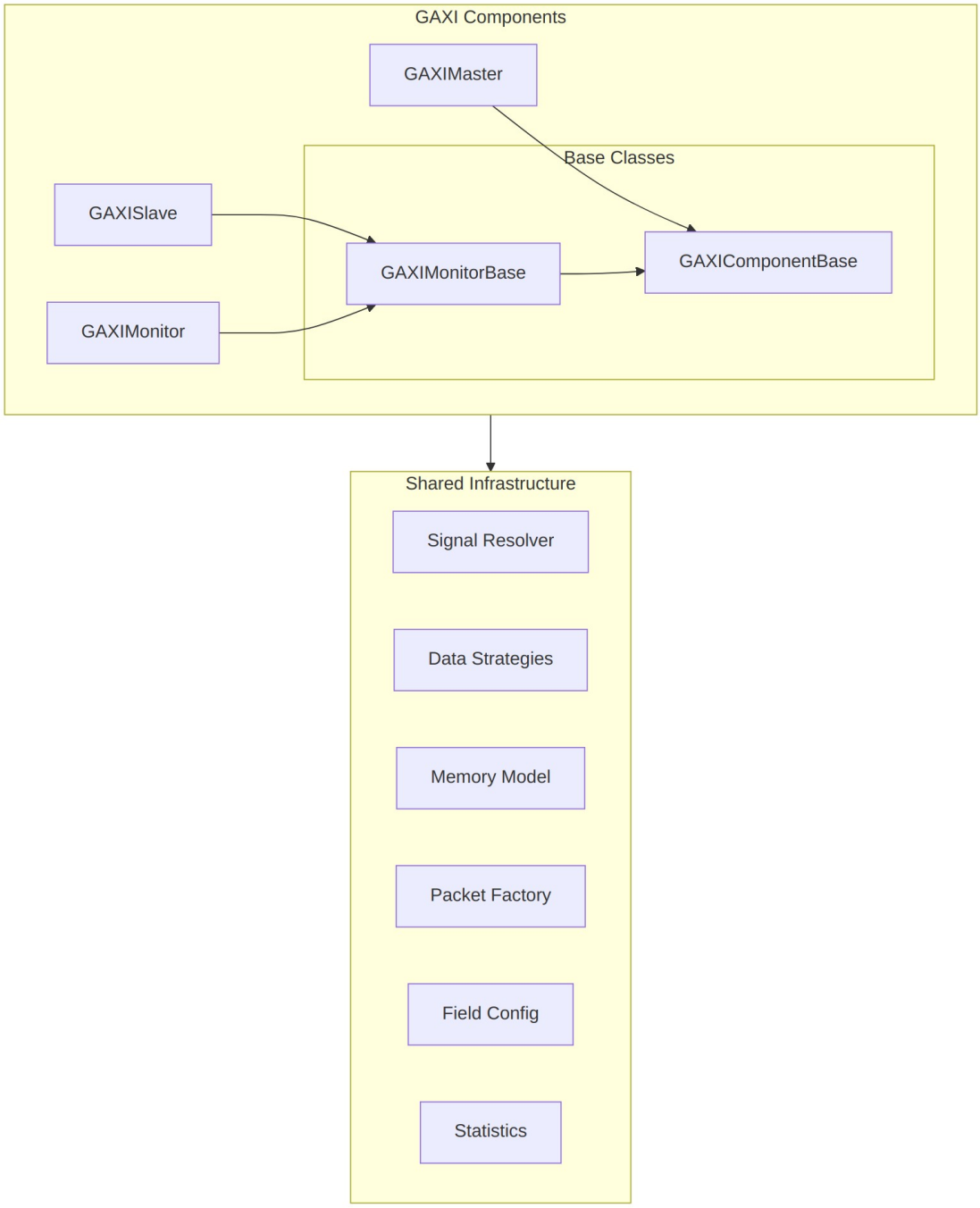


Diagram 1

11.2 Key Design Principles

11.2.1 1. Unified Base Classes

- **GAXIComponentBase**: Common functionality for all GAXI components

- **GAXIMonitorBase**: Shared monitoring capabilities for slaves and monitors
- Eliminates code duplication while preserving exact APIs

11.2.2 2. Performance Optimization

- Cached signal references for 40% faster data collection
- Thread-safe operation for parallel testing environments
- Optimized data strategies eliminate repeated signal lookups
- Pre-computed field validation rules

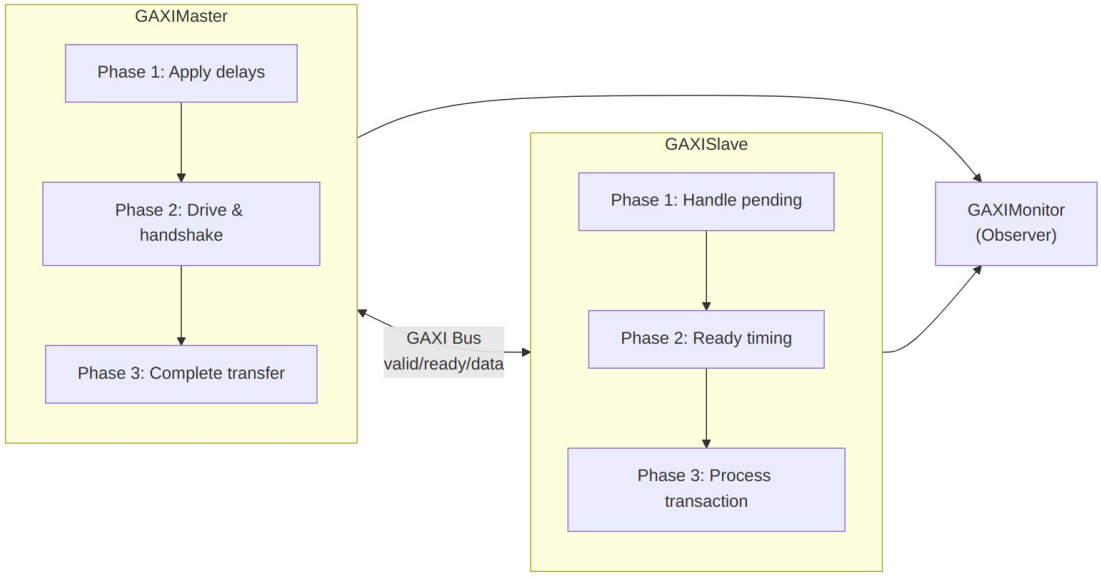
11.2.3 3. Enhanced Debugging

- Structured pipeline phases with state tracking
- Optional pipeline debugging with detailed transition logging
- Comprehensive statistics collection at multiple levels
- Protocol violation detection and reporting

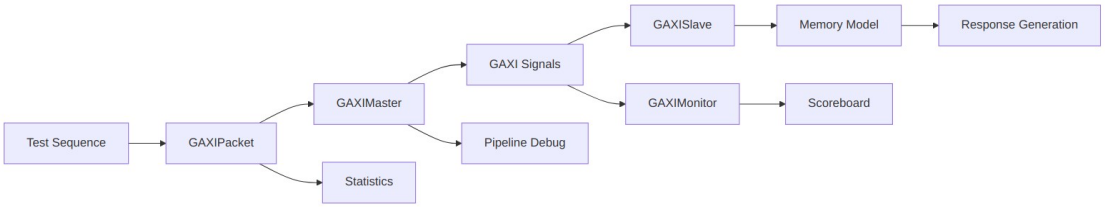
11.2.4 4. Flexible Configuration

- Automatic signal discovery with manual override capability
- Multi-signal and single-signal field modes
- Configurable timing and randomization patterns
- Memory model integration for transaction processing

11.3 Component Relationships



Master - Slave Communication



Data Flow Architecture

11.4 Signal Resolution System

The GAXI components use an advanced signal resolution system:

11.4.1 Automatic Discovery

- Pattern matching against DUT ports with parameter combinations
- Support for different signal naming conventions (`i_`/`o_` prefixes)
- Multi-signal mode for individual field signals
- Single-signal mode for combined data signals

11.4.2 Manual Override

```
signal_map = {
    'valid': 'master_valid_signal',
    'ready': 'slave_ready_signal',
    'data': 'transfer_data_signal'
}
component = GAXIMaster(dut, ..., signal_map=signal_map)
```

11.4.3 Mode Support

- **Single-signal mode**: All fields packed into one data signal
- **Multi-signal mode**: Individual signals for each field
- **Mixed mode**: Combination based on field configuration

11.5 Pipeline Architecture

11.5.1 GAXIMaster Pipeline

1. **Phase 1**: Apply configurable delays using randomizer
2. **Phase 2**: Drive signals and wait for ready handshake
3. **Phase 3**: Complete transfer and clean up signals

11.5.2 GAXISlave Pipeline

1. **Phase 1**: Handle pending transactions (deferred captures)

2. **Phase 2**: Apply ready delays and assert ready signal
3. **Phase 3**: Detect handshake and process transaction

11.5.3 Timing Modes

- **Skid mode**: Standard pipeline with immediate data capture
- **FIFO MUX mode**: Pipeline with immediate data capture
- **FIFO FLOP mode**: Pipeline with delayed data capture (slave side only)

11.6 Memory Integration

11.6.1 Unified Memory Operations

```
# Write to memory
success, error = component.write_to_memory_unified(packet)

# Read from memory
success, data, error = component.read_from_memory_unified(packet)
```

11.6.2 Memory Model Features

- High-performance NumPy backend
- Access tracking and coverage analysis
- Region management for logical organization
- Transaction-based read/write operations
- Boundary checking and validation

11.7 Statistics and Monitoring

11.7.1 Master Statistics

- Transaction throughput and latency metrics
- Protocol violation tracking
- Flow control and backpressure monitoring
- Error categorization and reporting

11.7.2 Slave Statistics

- Transaction acceptance rates and processing times
- Protocol compliance monitoring
- Memory operation tracking
- Pipeline efficiency metrics

11.7.3 Monitor Statistics

- Observed transaction counting
- Protocol violation detection
- X/Z signal violation tracking
- Coverage analysis

11.8 Randomization and Timing

11.8.1 FlexRandomizer Integration

```
# Constrained random delays
constraints = {
    'valid_delay': ([(0, 0), (1, 5), (10, 20)], [0.6, 0.3, 0.1]),
    'ready_delay': ([(0, 2), (3, 8)], [0.8, 0.2])
}
randomizer = FlexRandomizer(constraints)
```

11.8.2 Timing Profiles

- **Backtoback**: Zero delay for maximum throughput
- **Fast**: Mostly zero delay with occasional small delays
- **Constrained**: Balanced distribution for realistic testing
- **Stress**: Wide variation for corner case testing

11.9 Error Handling and Recovery

11.9.1 Pipeline Error Recovery

- Automatic signal cleanup on timeout or error
- Graceful degradation for missing signals
- Comprehensive error reporting with context
- Reset handling during operation

11.9.2 Protocol Validation

- Valid/ready handshake verification
- Signal value validation (X/Z detection)
- Timing constraint checking
- Memory boundary validation

11.10 Factory System

11.10.1 Simple Component Creation

```
# Create individual components
master = create_gaxi_master(dut, "Master", "", clock, field_config)
slave = create_gaxi_slave(dut, "Slave", "", clock, field_config)

# Create complete system
system = create_gaxi_components(dut, clock, field_config=field_config)
```

11.10.2 Test Environment Setup

```
# Complete test environment with defaults
env = create_gaxi_test_environment(dut, clock, data_width=32)
```

11.11 Performance Characteristics

11.11.1 Optimizations

- **40% faster data collection** through cached signal references
- **30% faster data driving** through cached driving functions
- **Thread-safe caching** for parallel test execution
- **Reduced memory overhead** through efficient data structures

11.11.2 Scalability

- Support for large field configurations (optimized initialization)
- Efficient memory usage in long-running tests
- Parallel test execution capabilities
- Resource-conscious operation

11.12 Integration Points

11.12.1 CocoTB Integration

- Standard BusDriver/BusMonitor inheritance
- Compatible with cocotb timing and event model
- Proper signal handling and lifecycle management
- Reset and clock domain handling

11.12.2 Framework Integration

- Shared component infrastructure utilization
- Scoreboard and transformer compatibility

- Statistics aggregation and reporting
- Test framework integration patterns

11.13 Advanced Features

11.13.1 Dependency Tracking

- Transaction dependency chains in sequences
- Completion-based dependency resolution
- Circular dependency detection
- Order enforcement for dependent transactions

11.13.2 Debug Capabilities

- Pipeline state visualization
- Signal transition logging
- Performance bottleneck identification
- Memory access pattern analysis

11.13.3 Extensibility

- Plugin architecture for custom behavior
- Callback systems for transaction processing
- Custom field configurations and packet types
- Protocol-specific extensions and modifications

The GAXI components provide a robust, high-performance foundation for AXI-like protocol verification while maintaining simplicity of use and comprehensive debugging capabilities.